

## GeoAgent: a hierarchical LLM-based multi-agent architecture for autonomous spatial analysis

Qingming Lin, Liuchang Xu, Sensen Wu, Ruichen Mao, Chao Wang, Hailin Feng, Bo Huang & Zhenhong Du

To cite this article: Qingming Lin, Liuchang Xu, Sensen Wu, Ruichen Mao, Chao Wang, Hailin Feng, Bo Huang & Zhenhong Du (16 Feb 2026): GeoAgent: a hierarchical LLM-based multi-agent architecture for autonomous spatial analysis, International Journal of Geographical Information Science, DOI: [10.1080/13658816.2026.2624784](https://doi.org/10.1080/13658816.2026.2624784)

To link to this article: <https://doi.org/10.1080/13658816.2026.2624784>



Published online: 16 Feb 2026.



Submit your article to this journal [↗](#)



Article views: 1220



View related articles [↗](#)



View Crossmark data [↗](#)



RESEARCH ARTICLE



# GeoAgent: a hierarchical LLM-based multi-agent architecture for autonomous spatial analysis

Qingming Lin<sup>a</sup> , Liuchang Xu<sup>a,b</sup> , Sensen Wu<sup>b</sup> , Ruichen Mao<sup>c</sup>,  
Chao Wang<sup>b</sup>, Hailin Feng<sup>a</sup>, Bo Huang<sup>d</sup> and Zhenhong Du<sup>b</sup>

<sup>a</sup>School of Mathematics and Computer Science, Zhejiang Agriculture and Forestry University, Hangzhou, China; <sup>b</sup>School of Earth Sciences, Zhejiang University, Hangzhou, China; <sup>c</sup>Moganshan Geospatial Information Laboratory, Huzhou, China; <sup>d</sup>Department of Geography, The University of Hong Kong, Hong Kong, China

## ABSTRACT

Large language models (LLMs) provide powerful impetus for spatial analysis tasks in GIS. LLM-based agents demonstrate immense potential to revolutionize traditional spatial analysis workflows. However, designing a multi-agent collaborative architecture that can achieve autonomy for complex spatial analysis, while overcoming the limitations of traditional GIS in flexibility and dynamic adaptability, remains a critical challenge. This paper proposes GeoAgent, a hierarchical multi-agent collaborative spatial analysis framework that enhances automation, intelligence, and reliability in complex spatial analysis. Its three-layer architecture comprises: the Planning Layer that parses requirements and formulates execution plans; the Execution Layer that schedules execution agents through a manager agent; and the Review Layer that achieves autonomous verification and optimization. The framework integrates three core toolsets enabling environmental perception and data insight capabilities. Experiments across 147 spatial analysis tasks show GeoAgent achieved 94.56% and 95.24% success rates using GPT-4o and DeepSeek-V3 configurations, respectively. Ablation studies confirmed the critical role of Planning and Review Layers in ensuring workflow integrity and result reliability for complex tasks. GeoAgent contributes a novel paradigm for automated intelligent spatial analysis while validating hierarchical multi-agent collaboration and self-verification mechanisms for geospatial challenges, providing design insights for next-generation autonomous GIS systems.

## ARTICLE HISTORY

Received 26 May 2025  
Accepted 27 January 2026

## KEYWORDS

Large language model;  
multi-agent framework;  
spatial analysis;  
autonomous gis; GeoAI

## 1. Introduction

In recent years, the integration of geographic information science and Artificial Intelligence (AI) has catalyzed the rapid development of Geospatial Artificial Intelligence (GeoAI). Amidst the wave of Generative AI, large language models (LLMs), with their remarkable capabilities in text generation, comprehension, and reasoning,

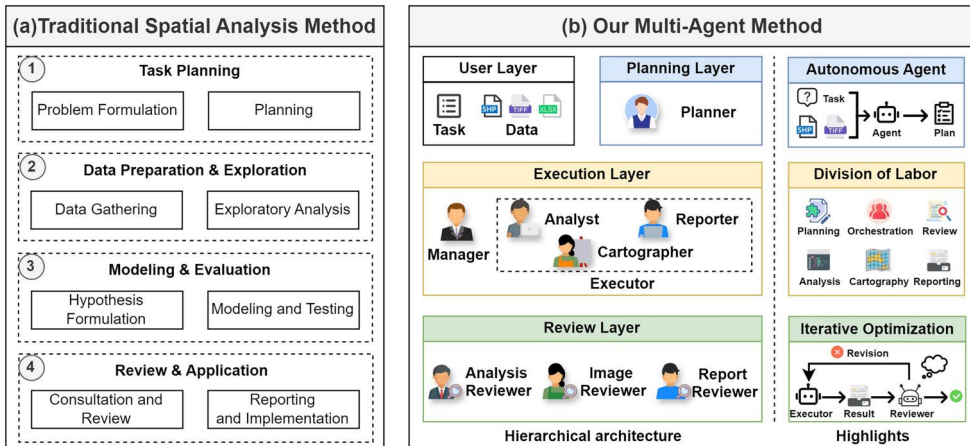
are progressively permeating both general-purpose and specialized domain applications. Numerous studies have explored the potential applications of LLMs in the GIS field, evaluating their performance in handling various GIS tasks such as GIS-based question answering, geospatial code generation, and geolocating (Hou *et al.* 2024c, Feng *et al.* 2024, Xu *et al.* 2025). These studies indicate that LLMs demonstrate strong generalization abilities in the GIS field and can effectively address spatial tasks of considerable complexity.

To expand the capability boundaries of LLMs, researchers have shifted their focus to LLM-based agents. Through prompt engineering, tool calling, memory management, and other techniques, agents can demonstrate more powerful task execution capabilities, especially in specific domains or for specific tasks (Xi *et al.* 2025). Within the agent framework, multi-agent system design is particularly prominent. It involves the collaborative efforts of agents to decompose complex tasks and assign them to different specialized agents, thereby forming more sophisticated and intelligent task-processing systems that enhance system diversity and efficiency. In the GIS field, research has begun to combine agents with specific GIS tasks, exploring how to utilize the agent framework to solve more complex challenges, such as automatic acquisition of GIS data and automated processing of Shapefiles (Ning *et al.* 2025, Lin *et al.* 2025).

In the interdisciplinary research area of LLMs and GIS, spatial analysis tasks are undoubtedly one of the most compelling applications. Spatial analysis is not only a core technology in the GIS field but also an indispensable analytical tool for many related disciplines. However, the high technical threshold and complex operational workflows of traditional GIS spatial analysis toolchains present significant barriers for researchers without professional backgrounds. LLM-based agent systems, through natural language interaction interfaces and intelligent workflow management, can effectively lower these technical barriers. This enables researchers from various fields to focus on their disciplinary problems rather than technical intricacies, thereby democratizing spatial analysis capabilities and fostering their widespread adoption. Current research has preliminarily explored the potential applications of LLMs in supporting spatial analysis tasks, such as using LLMs to automatically execute spatial analysis and generate geospatial code (Lin *et al.* 2025, Hou *et al.* 2024d).

Despite promising prospects, applying LLM-based agents directly to complex, real-world spatial analysis tasks still faces numerous challenges. Spatial analysis typically requires integrating multi-source heterogeneous data, executing multi-step analytical workflows, invoking various specialized tools, and performing complex spatial reasoning. However, existing agent architectures still lack systematic design to address the complexity of spatial analysis, particularly in establishing an integrated framework that unifies task planning, analytical process execution, and result validation. There remains a lack of in-depth research on how to construct an LLM-based agent framework capable of handling diverse spatial analysis tasks.

To effectively address these challenges, hierarchical multi-agent architectures emerge as a promising solution. This architecture simulates the collaborative patterns of human expert teams in traditional spatial analysis by establishing hierarchical divisions of responsibilities and authorities, enabling effective separation of planning, execution, and validation. This structured design not only decomposes complex tasks into



**Figure 1.** Conceptual diagram of the GeoAgent framework. (a) illustrates the conventional, sequential process of spatial analysis. (b) introduces our hierarchical multi-agent architecture, which reorganizes the workflow into three collaborative layers.

a series of subtasks handled by specialized units, but also provides an organizational foundation for introducing quality control mechanisms, thereby systematically enhancing the robustness and reliability of complex task processing.

In summary, addressing the potential and challenges of LLM-based agents in GIS applications, this paper presents GeoAgent. As illustrated in Figure 1, this framework is designed as a hierarchical multi-agent collaborative spatial analysis system designed to solve automation and intelligence challenges in complex spatial analysis tasks. As an important exploration in autonomous GIS research, the main contributions of this study are reflected in three aspects:

- We propose a novel hierarchical spatial analysis framework that demonstrates superior performance and adaptability across diverse datasets and analytical scenarios in spatial analysis tasks.
- To address the research gap in self-verification within autonomous GIS systems, we develop an innovative Review Layer architecture. This explores the implementation of self-verification for spatial analysis tasks, establishing a methodological framework that enhances the reliability of automated spatial analysis outputs.
- We validate the effectiveness and feasibility of a divide-and-conquer, team-based collaborative approach for spatial analysis tasks in GIS. This extends the current knowledge by introducing novel agent organizational models and collaborative paradigms for addressing complex geospatial problems.

## 2. Related works

Recently, the significant advancements of LLMs have become a key driving force in the field of AI, primarily attributed to their advanced capabilities in natural language understanding, text generation, and complex reasoning (Achiam *et al.* 2023, Team *et al.* 2023, Touvron *et al.* 2023, Liu *et al.* 2024). LLM-based agents have further

expanded the application boundaries of these models, enabling them to actively plan, execute tasks, and interact with the environment or other tools, thereby demonstrating immense potential for solving complex problems in various domains (Yao *et al.* 2023, Ning *et al.* 2025, Wu *et al.* 2023, Hong *et al.* 2024, Wang *et al.* 2024). By combining the cognitive capabilities of LLMs with the functionalities of external tools, these agents provide new paradigms for automated and intelligent processes.

The rapidly growing number of related publications demonstrates the vibrant development and potential value of this interdisciplinary field (Mai *et al.* 2025). Research in Autonomous GIS has initially explored how to apply LLMs to spatial analysis and clearly defined five core objectives of autonomous GIS: self-generating, self-organizing, self-verifying, self-executing, and self-growing (Li and Ning 2023). This research introduced the LLM-Geo proof-of-concept prototype, capable of decomposing complex tasks into sequential data operation workflows represented as Directed Acyclic Graphs. This demonstrated that the fusion of LLMs with GIS can automate spatial analysis and provide a more accessible technological pathway for non-GIS professionals. Meanwhile, LLM-Find was proposed as an automated geospatial data retrieval framework that intelligently selects appropriate data sources, generates and executes code, and utilizes LLMs for error debugging (Ning *et al.* 2025). These research efforts provide valuable theoretical foundations and technical references for the design of autonomous GIS frameworks.

GIS relies on specialized tools to support research and applications, and leveraging LLMs to enhance the automation and intelligence of tool use within traditional GIS workflows has become a significant research direction (Qu *et al.* 2025, Chen *et al.* 2025). GeoGPT leverages LLMs to interpret users' natural language queries, plan geospatial workflows, and precisely call appropriate GIS functions such as data crawling, spatial queries, facility location, and cartography (Zhang *et al.* 2024c). MapGPT integrates LLMs with professional cartographic tools to achieve automation in the map creation process (Zhang *et al.* 2024a). GeoTool-GPT significantly enhances the ability of LLMs to master GIS tools by fine-tuning LLaMA-2-7B on a domain-specific instruction dataset (Wei *et al.* 2025). The GTChain framework utilizes a seed task-guided self-instruct strategy to generate a geospatial tool-use instruction tuning dataset. After fine-tuning, it can autonomously understand geospatial tasks and generate tool-use chains (Zhang *et al.* 2024b). GIS Copilot, an assistant directly integrated into QGIS, enables natural language-driven spatial analysis and geoprocessing using QGIS functionalities (Akinboyewa *et al.* 2025). These studies indicate that the integration of LLMs and GIS tools has achieved initial success, and the tool calling capabilities of LLMs can be generalized to the GIS domain through certain strategies.

The remarkable proficiency of LLMs in code generation has spurred extensive exploration by researchers into their application in geospatial code generation (Jiang *et al.* 2024, Chen *et al.* 2021). GeoCode-Eval proposed a comprehensive evaluation framework to assess the capabilities of LLMs in geospatial code generation, effectively addressing key challenges such as domain knowledge gaps and coding hallucinations (Hou *et al.* 2024c). GeoCode-GPT was proposed as the first LLM specifically designed for geospatial code generation; by fine-tuning Code Llama-7B with a specialized dataset, it addresses common issues like coding refusal and hallucinations (Hou *et al.* 2024d). At the same time, training-free strategies aimed at enhancing model reliability

without incurring the high costs of fine-tuning have also become a significant focus for researchers. The Chain-of-Programming framework effectively addressed code hallucination issues by dividing the geospatial code generation process into five sequential stages—requirement analysis, algorithm design, code implementation, code debugging, and code annotation (Hou *et al.* 2024a). GEE-OPs introduced a geospatial operator knowledge base that significantly enhanced LLM-driven Google Earth Engine code generation capabilities (Hou *et al.* 2024b). Furthermore, other research has expanded the capabilities of existing geospatial code generation frameworks by incorporating dynamic, multi-step processes that not only generate code but also validate it through comprehensive review procedures (Chen *et al.* 2024). Similarly, GeoCogent was proposed as an LLM-based agent framework, which integrates planning, tool-augmented reasoning, and memory mechanisms to tackle core challenges in geospatial coding (Hou *et al.* 2025). These studies demonstrate that code generation has shown tremendous potential in GIS, providing powerful technical support for the automation of spatial analysis.

In summary, we note that while the application of LLMs in GIS, particularly in areas such as geospatial code generation and the call of specific GIS tools, has shown encouraging progress and achieved preliminary success, there remains significant room for exploration and optimization potential in constructing a systematic agent framework capable of efficiently handling complex spatial analysis tasks. The hierarchical multi-agent framework GeoAgent proposed in this paper represents an innovative attempt that validates the feasibility and effectiveness of hierarchical collaborative mechanisms in the field of spatial analysis. In terms of self-verification, our designed Review Layer provides a valuable reference paradigm for autonomous GIS research. The introduction of the GeoAgent framework not only enriches the forms of LLM-based agent applications in the GIS domain but also provides a foundation for subsequent research to explore a wider array of possible framework structures.

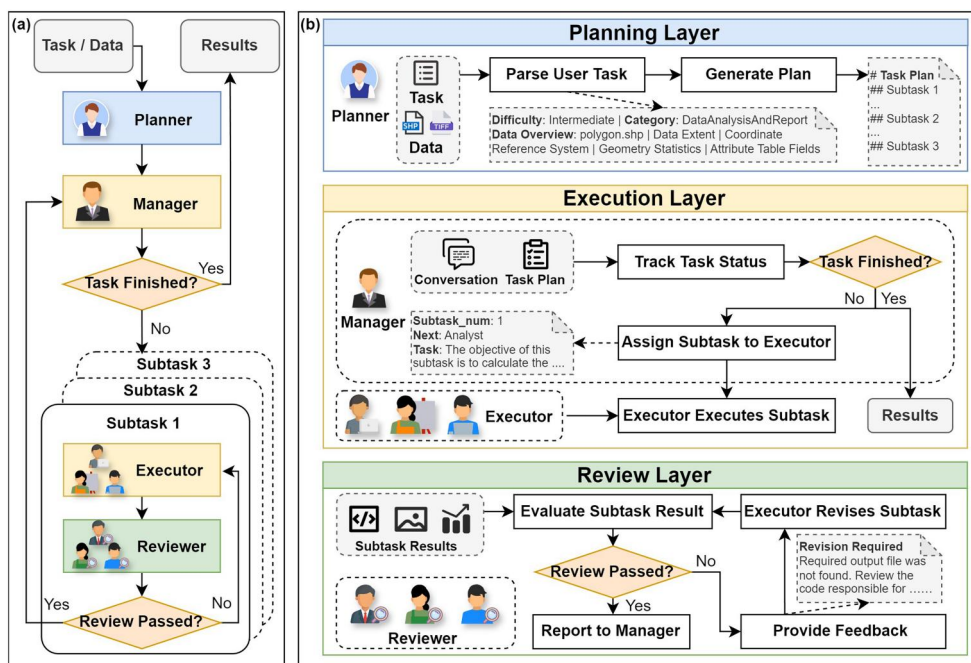
### 3. Methodology

#### 3.1. Hierarchical multi-agent framework

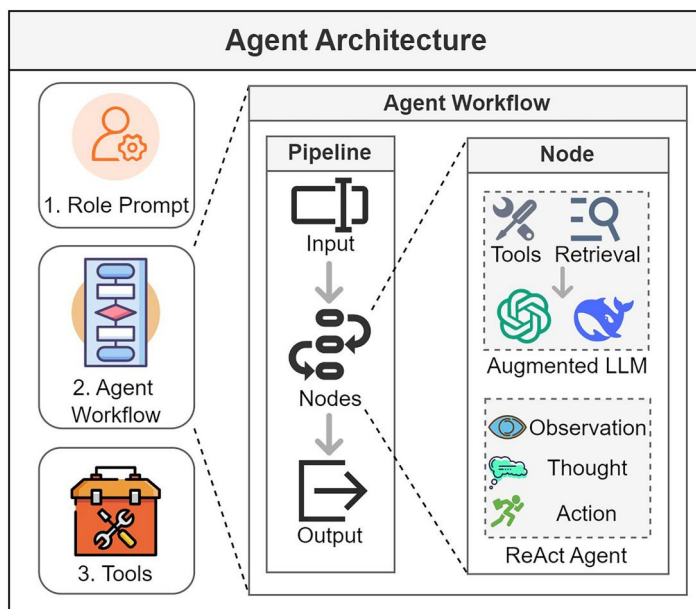
The GeoAgent framework optimizes traditional spatial analysis workflows using LLM-based agents. Based on an in-depth analysis of GIS spatial analysis, we have designed a hierarchical multi-agent collaborative spatial analysis framework. As illustrated in Figure 2, this framework is composed of a Planning Layer, an Execution Layer, and a Review Layer. The framework supports multimodal spatial data input including vector and raster data, and can process spatial analysis tasks of varying complexity. Its flexible data integration mechanism ensures the system's widespread applicability.

##### 3.1.1. Agent components

As illustrated in Figure 3, based on the comprehensive requirements of geospatial analysis tasks for domain knowledge, professional tools, and complex reasoning, we abstract each agent as a unified structure comprising three core components: the role prompt, the agent workflow, and the tools. This design paradigm ensures that



**Figure 2.** Workflow decomposition in the GeoAgent framework. The diagram illustrates the framework's operational flow at two levels of abstraction. (a) A simplified flowchart showing the high-level, cyclical interaction among the core agent roles (Planner, Manager, Executor, and Reviewer). (b) A detailed view of the concrete processes and information exchange within and between the three layers, highlighting key mechanisms.



**Figure 3.** The unified architecture of an individual agent. Each agent is composed of three core components: role prompt, agent workflow, and tools.

different agents within the framework possess a consistent architecture and standardized implementation mechanisms, thereby supporting effective collaborative operations.

The role prompt defines an agent's responsibility boundaries and interaction context. Through explicit role settings, goal constraints, and output rules, the prompt enhances the controllability and consistency of the model's generation process. For example, in the Execution Layer, prompts for different agents emphasize their respective professional focuses, achieving functional complementarity based on the same underlying language model capabilities.

The agent workflow adopts a directed graph structure composed of several nodes. When the system executes a spatial analysis task, all agents operate within a shared workspace, collaborating by reading and generating intermediate products. These products include multimodal content such as system messages, code, images, and data files, ensuring the traceability and transparency of the multi-agent collaboration. We employ the LangGraph open-source framework to design modular processing workflows with nodes as the fundamental units, and we construct differentiated pipelines tailored to various specialized roles. The nodes are classified into two types: one is the tool-augmented LLM node, capable of invoking external tools as needed during its reasoning process; the other is the ReAct agent node, which advances tasks by following the thought-action-observation loop. Both node types produce intermediate products as output, enabling flexible combinations of different nodes. To enhance execution stability and efficiency, the system sets an iteration limit for each node to prevent excessive looping during the reasoning process.

Tools provide a unified interface for the interaction between nodes and the external environment. Each node can access its authorized tools during runtime and obtain corresponding functional descriptions and invocation methods. These tools include not only general-purpose interfaces for basic tasks such as file operations and data I/O but also specialized functions for parsing and understanding geospatial data. All tools are executed within a sandbox environment to ensure the security of the invocation process. The specific composition and implementation details of the tools will be further elaborated in [Section 3.2](#).

### 3.1.2. Planning layer

The spatial analysis process follows several clearly defined stages, beginning with problem identification and outline planning (de Smith *et al.* 2018). Building on this principle, we designed the Planning Layer, whose core component is the Planner agent. This agent is responsible for decomposing a user's high-level analysis requirements into manageable subtasks and generating a corresponding execution plan.

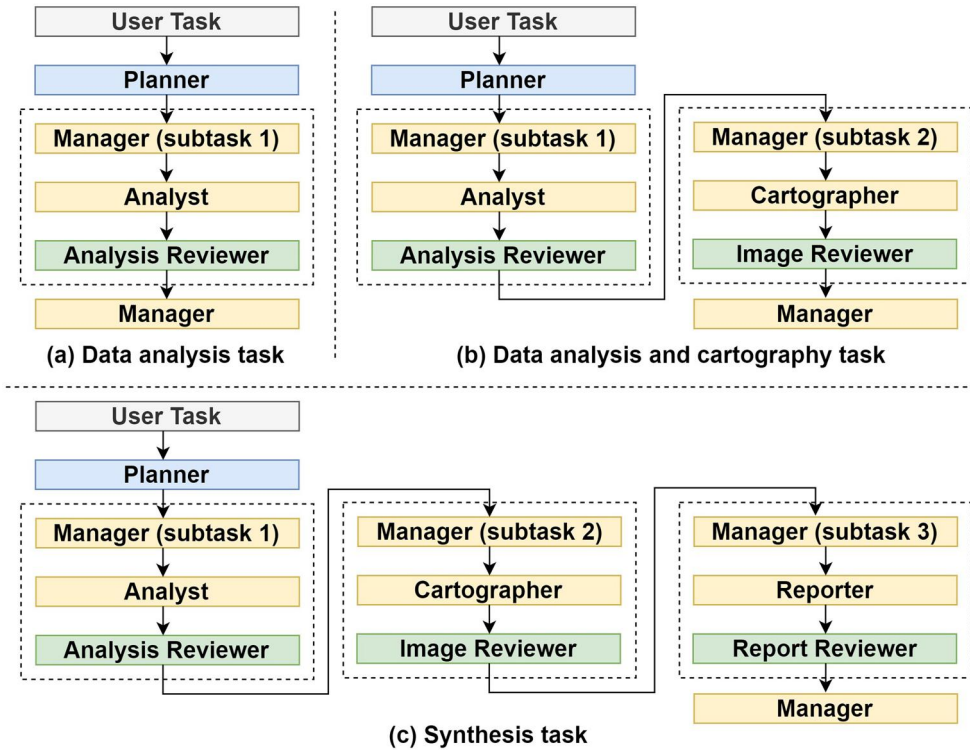
The Planner agent's workflow consists of two nodes: task parsing and task planning. The task parsing node receives the user's task description and spatial data, utilizing LLMs for multi-dimensional analysis. This node evaluates task complexity, conducts category identification, determines required data types, and performs semantic analysis to transform complex tasks into manageable subtask units. This parsing process can be represented as:

$$\{C, A, P, S\} \leftarrow \text{Parse}(I), \quad (1)$$

where  $I = \{T, D\}$  represents input information, including the user's task description  $T$  and uploaded spatial data list  $D$  (e.g., vector data, raster data, and tabular data). The output parameter  $C$  represents task complexity;  $A$  represents task category;  $P$  represents the data type currently requiring processing; and  $S$  represents the list of subtasks.

The task complexity and task category are first passed to the task planning node and incorporated as contextual information into the prompt for that node, guiding the planning node to generate a solution that matches the initial task assessment in both structural and detailed levels. Concurrently, this information is also encapsulated within the final plan to provide decision support for the Manager agent during subsequent scheduling. The task complexity is categorized into three levels based on an overall difficulty assessment—basic, intermediate, and advanced—which helps the Manager agent estimate potential resource consumption. The task category is classified into one of six types based on the expected output: data analysis, cartography, data analysis and cartography, data analysis and report, cartography and report, or synthesis. As illustrated by the representative workflows in Figure 4, these categories correspond to different combinations of the three agent roles in the Execution Layer, providing the Manager agent with a reference for configuring the appropriate execution agents.

Subsequently, the task planning node translates the abstract understanding derived from the parsing stage into a concrete execution plan. Guided by the contextual



**Figure 4.** Workflow diagrams illustrating three of the six task categories defined in our framework.

information received from the task parsing phase, the node invokes a plan document generation tool to construct the plan. The final generated plan is a well-structured document that explicitly defines for each subtask its specific objective, required input dependencies, and expected outputs. This process can be represented as:

$$Plan \leftarrow \text{Planner}(C, A, P, S). \quad (2)$$

The workflow of the Planning Layer is designed to decouple task comprehension from plan design, enabling the system to address diverse spatial analysis requirements with greater flexibility. The task parsing node focuses on understanding what to do, while the task planning node concentrates on how to do it. This separation of functions effectively prevents a single node from bearing an excessive cognitive load. As illustrated in the workflow diagram in [Figure A1](#), these two components operate sequentially to transform a user instruction into a structured execution plan.

The role prompt for the Planner agent clarifies its responsibility boundaries, constraining its behavior to task parsing and plan generation, rather than specific execution or result presentation. The prompt also defines output specifications, requiring the Planner to produce a complete task parsing result and a detailed execution plan. Furthermore, we incorporate few-shot examples that demonstrate typical parsing patterns for tasks of varying complexities and categories, providing classification guidance for the model.

The Planner relies on specialized tools to perform its duties. During the task parsing phase, it employs a data understanding tool to analyze the input data. Subsequently, in the task planning phase, it utilizes a document generation tool to construct a structured execution plan. The implementation details of these tools are described in [Section 3.2](#). The final execution plan follows a predefined Markdown template, provided in [Appendix A.1](#). The template specifies fixed fields such as the task overview, input data description, subtask sequence, and output requirements, and defines the structure in which these elements should be presented.

Through the synergy of meticulously designed prompts, workflows, and specialized tools, the Planner can dynamically match the complexity of the execution plan to the difficulty of the task. For simple, single-step tasks (e.g., buffer analysis), the system generates a streamlined execution plan that avoids unnecessarily complex processing. Conversely, for complex, multi-stage tasks (e.g., site selection analysis under multiple constraints), it constructs a comprehensive, multi-level solution to ensure the logical integrity of the analysis workflow.

### 3.1.3. Execution layer

The Execution Layer design adheres to cognitive science principles of task decomposition and professional collaboration, constructing a distributed processing architecture centered around a Manager agent with multiple specialized execution agents working collaboratively. As Simon pointed out in his theory of bounded rationality, when facing complex problems, clear task division and specialized collaboration can effectively overcome individual limitations in information processing and decision-making (Simon 1997). Based on this theoretical insight, we designed an Execution Layer architecture comprising a Manager agent for overall coordination, and three types of specialized

execution agents: Analyst, Cartographer, and Reporter. This forms a complete workflow encompassing spatial data processing, visual representation, and report generation.

**3.1.3.1. Manager agent.** The Manager agent serves as the core coordinator of the Execution Layer, responsible for the critical task of translating the execution plan from the Planning Layer into executable subtasks and orchestrating the collaboration among execution agents. This can be represented as:

$$(S_t, R_t) \leftarrow \text{Manager}(Plan, T_{\text{Progress}}, R_{\text{Info}}), \quad (3)$$

where  $S_t$  denotes the subtask to be executed at the current moment,  $R_t$  represents the professional role assigned to execute this subtask. Each of the three input types has a specific source and function:

- *Plan* represents the output from the Planner agent. The Manager agent acquires this detailed execution plan by invoking a plan reading tool. The plan contains the execution sequence for subtasks and the specifications for their expected outputs, providing the Manager with systematic guidance for task allocation.
- $T_{\text{Progress}}$  represents the current task execution progress, which is composed of intermediate products within the workspace. These products include interactive messages between execution agents, generated code files, data visualizations, and analysis results, all of which reflect the current state and achieved outcomes of the task.
- $R_{\text{Info}}$  signifies the capability profiles of each specialized role in the Execution Layer, provided to the Manager agent through system prompts. These prompts detail the responsibility boundaries, core capabilities, and applicable scenarios of the three execution agents: Analyst, Cartographer, and Reporter. This allows the Manager to schedule based on role specializations when assigning tasks.

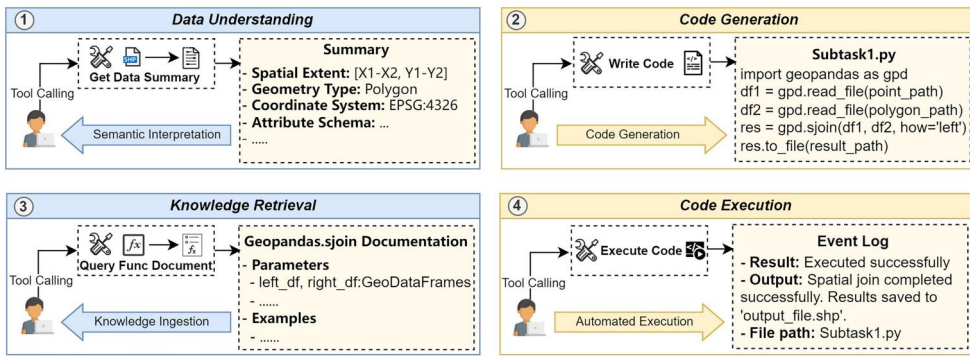
The Manager agent is driven by a tool-augmented LLM. The plan reading tool enables it to accurately interpret the output from the Planner agent, while role prompts constrain its generative behavior to focus on subtask selection and role assignment, rather than involvement in the execution of analysis or visualization. Based on its assessment of execution progress, the Manager effectively allocates subtasks and oversees the collaborative workflow. Specialized execution agents receive instructions and generate corresponding outputs, which can be represented as:

$$O_t \leftarrow \text{Executor}(S_t), \quad (4)$$

where  $O_t$  represents the output object, such as code or charts.

Orchestrated by the Manager agent, the three specialized agents of the Execution Layer form a functionally complementary and collaboratively operating whole. Given the inherently complex attribute structures and diverse spatial relationships, this clear professional division of labor and collaboration mechanism enables each agent to focus on its core area of responsibility.

**3.1.3.2. Analyst agent.** As illustrated in [Figure 5](#), the Analyst agent serves as the specialized execution component for spatial data processing and analysis and is



**Figure 5.** The four-stage workflow of the Analyst agent, showcasing the integration of cognitive processes and specialized tool calls.

architecturally designed based on the advanced capabilities of LLMs in code generation and execution. However, existing LLMs are predominantly trained on general corpora and exhibit significant limitations in understanding the specific function syntax, parameter configurations, and data structures within geospatial computational ecosystems, consequently restricting their efficacy in handling GIS-specific tasks (Hou *et al.* 2024c).

To address this challenge, this study constructs an Analyst agent system with a four-step workflow that integrates data understanding, code generation, code inspection, and code debugging. This workflow employs a hybrid node architecture: the first two steps are driven by tool-augmented LLM nodes, while the latter two are implemented using ReAct agent nodes.

In the data understanding and code generation stages, the tool-augmented LLM node analyzes the input geospatial data by calling a data understanding tool, outputting a detailed summary containing information like data type, spatial attribute characteristics, and coordinate reference system, providing data information for subsequent code generation. Based on these data insights, the node in the code generation stage generates initial analysis code, constructing the analysis logic that meets the task requirements. This stage focuses on converting the task requirements into Python code and does not involve code debugging or error correction.

The code inspection stage uses a ReAct agent node to implement syntax checks and function conformity evaluation. The node's role prompt guides it to call the code reading tool and a function documentation query tool to check code quality from multiple dimensions, including file path correctness, function parameter reasonability, data type matching, and logical structure integrity. This node generates a detailed inspection message, outputting the quality assessment result for the code file.

The ReAct agent node in the code debugging stage is responsible for iterative optimization of the code based on the inspection results. The node's role prompt is designed to guide it to continuously iterate the code until it fully meets the subtask requirements and is free of execution errors. Through the reasoning mechanism of the ReAct architecture, the node observes the code execution results, analyzes existing problems, and then calls a code execution tool to perform targeted optimization. This process continues until the code reaches the expected functional completeness.

**3.1.3.3. Cartographer agent.** The Cartographer agent, as the specialized execution component for geospatial data visualization, is architecturally designed around the workflow of transforming spatial data into visual representations. Following the same code generation approach as the Analyst agent, the Cartographer also employs a four-stage hybrid node architecture, but its core difference lies in the specialized positioning of the node's role prompts.

The role prompt for the Cartographer explicitly defines it as a geospatial visualization expert, emphasizing the need to comprehensively consider data semantic characteristics, spatial relationship patterns, and principles of visual communication. The prompt constrains its workflow as follows: in the data understanding stage, it evaluates the input data's spatial distribution characteristics, attribute structure, and spatial scale characteristics; based on the data understanding results, in the code generation stage, it selects visualization types and symbology systems that match the data's characteristics, and generates visualization code that includes layer construction, symbol configuration, and layout settings; in the code inspection stage, it retrieves the specifications for relevant cartographic functions; in the code debugging stage, it enhances the visualization effect by analyzing colors and adjusting visual hierarchy in the code. By analyzing data characteristics to guide the choice of visualization methods, the Cartographer avoids the indiscriminate use of standard templates, thereby enabling its visualization results to more effectively convey the core information of the data.

**3.1.3.4. Reporter agent.** The Reporter agent, serving as the specialized report generation component, is responsible for transforming the analytical results and visual images from other execution agents into structured, professional markdown-formatted report documents. Given that the report generation task is relatively clear and the process is relatively straightforward, the Reporter uses a simplified single-node architecture, driven by one ReAct agent node and equipped with a specialized document writing tool. Its role prompt positions it as a technical report writing expert, emphasizing the logical organization of information hierarchy, the accurate use of professional terminology, and the clear expression of content logic.

The Reporter follows the thought-action-observation loop of the ReAct framework to generate the report. In the observation stage, the Reporter systematically collects and analyzes the outputs from the Analyst and Cartographer, including analysis data, statistical results, and visual charts; in the thought stage, it organizes content and plans the structure based on the professional requirements of report writing; in the action stage, it calls the document writing tool to complete the content writing, implementing paragraph layout and the arrangement of text and figures. Through this specialized design, the Reporter can transform complex analysis results into a markdown-formatted report that is both academically rigorous and highly readable, providing the user with a complete summary of the spatial analysis results.

### 3.1.4. Review layer

Spatial analysis tasks are characterized by high specialization and complexity, with their result quality directly affecting decision-making effectiveness. Self-verification in autonomous GIS systems is a critical yet often overlooked component by researchers.

To address this, we designed the Review Layer, which adopts a specialized review structure corresponding to the Execution Layer, establishing a closed-loop quality control process that achieves systematic evaluation and feedback for the outputs from the Execution Layer.

The Review Layer is configured with three types of specialized reviewers, each corresponding to the outputs of the three roles in the Execution Layer: the Analysis Reviewer, the Image Reviewer, and the Report Reviewer. Each reviewer agent is composed of a ReAct agent node and is guided by a specific prompt for its review behavior and professional standards. After completing an evaluation, each reviewer generates a review decision and review comments, which can be represented as:

$$(P_t, F_t) \leftarrow \text{Reviewer}_i(O_t), \quad (5)$$

where  $O_t$  represents the object being reviewed (code, image, or report),  $P_t$  represents the binary decision to pass or revise, and  $F_t$  represents specific review feedback.

Specifically, the Analysis Reviewer focuses on evaluating the execution effectiveness of spatial analysis code. By utilizing code reading tools and file inspection tools, this reviewer examines code logic, algorithm selection, and execution efficiency, and validates the effectiveness of the generated data. The Image Reviewer leverages the text-image understanding capabilities of multimodal models to evaluate the quality of visualization products, ensuring effective communication of spatial information. This reviewer first extracts user requirements and task needs, then assesses visualization results from multiple dimensions, including the rationality of symbol selection, effectiveness of color schemes, clarity of visual hierarchy, and overall design aesthetics. For image review specifically, the process is:

$$(P_t, F_t) \leftarrow \text{ImageReviewer}(\text{VLM}(O_{\text{img}}, S_t)), \quad (6)$$

where VLM represents a large model with multimodal understanding capabilities, and  $S_t$  denotes the current task's requirements for visualization expression. The Report Reviewer focuses on evaluating the quality of analysis reports, primarily from three aspects: content completeness, logical consistency, and format accuracy. The content completeness check verifies whether the report includes all analytical results in data or image formats. The logical consistency check validates whether the arguments in the report are self-consistent and conform to the scientific principles of spatial analysis. The formatting accuracy check assesses whether the report is correctly written according to markdown syntax.

When a reviewer issues a revision decision, the feedback is sent to the corresponding agent in the Execution Layer, which is then prompted to make modifications. Conversely, if the output is approved, the result is sent to the Manager agent to determine the next subtask for execution. This process can be formally expressed as:

$$(S_{t+1}, R_{t+1}) \leftarrow \text{Manager}(\{P_t, F_t\}, \text{Plan}, T_{\text{Progress}}, R_{\text{info}}). \quad (7)$$

This mechanism establishes a revision loop, as presented in [Algorithm 1](#), wherein a revise decision triggers the re-assignment of the task back to the Execution Layer for correction. To prevent the system from entering inefficient or endless revision cycles, we introduced two constraints. First, we impose a ceiling on the number of revisions, which is set to one in this study. Second, we constrain the review criteria within the

role prompts, directing reviewers to prioritize whether the output meets the user's fundamental requirements and task objectives. For example, the acceptance criteria for the Analysis Reviewer prioritize the effectiveness and reliability of the code over achieving maximal computational efficiency.

Based on the coordination of the three layers described above, the complete execution procedure of GeoAgent is formally summarized in [Algorithm 2](#). To ensure stable system behavior across both algorithms, we introduce a set of execution control parameters. These include the maximum subtask revision count and the global execution step limit, which define the operational boundaries for the revision loop in [Algorithm 1](#) and the full execution pipeline in [Algorithm 2](#). The parameters and their functional roles are summarized in [Table A1](#).

---

**Algorithm 1.** Subtask Execute-Review Loop
 

---

```

1: Input: Current subtask  $S_t$ , assigned role  $R_t$ , revision limit  $R_{\max}$ 
2: Output: Executor's output  $O_t$ , decision  $P_t$ , feedback  $F_t$ , count  $n_{iter}$ 
3:  $n_{iter} \leftarrow 0$  // Initialize local iteration counter
4: // Loop until approved or max revisions reached
5: while  $n_{iter} < R_{\max}$  do
6:    $O_t \leftarrow \text{Executor}[R_t].\text{Execute}(S_t)$  // Assigned agent executes the subtask
7:    $role_{rev} \leftarrow \text{GetReviewer}(R_t)$  // Identify corresponding reviewer role
8:    $(P_t, F_t) \leftarrow \text{Reviewer}[role_{rev}].\text{Review}(O_t, S_t)$  // Reviewer evaluates output validity
9:    $n_{iter} \leftarrow n_{iter} + 1$  // Increment iteration count
10:  if  $P_t == \text{PASSED}$  then
11:    break // Exit loop if quality standards met
12:  end if
13: end while
14: return  $(O_t, P_t, F_t, n_{iter})$  // Return results to Manager
  
```

---



---

**Algorithm 2.** General execution procedure of GeoAgent
 

---

```

1: Input: User task  $T_{user}$ , user data  $D_{user}$ , max steps  $N_{\max}$ 
2: Output: Final results  $\mathcal{R}_{final}$ 
3:  $N_{steps} \leftarrow 0$  // Initialize global step counter
4: // Phase 1: Planning Layer
5:  $\{C, A, P, S\} \leftarrow \text{Planner.Parse}(T_{user}, D_{user})$  // Parse task: complexity, category,
   data type, subtask list
6:  $Plan \leftarrow \text{Planner.Generate}(C, A, P, S)$  // Generate execution plan
7:  $\text{Workspace.Update}(Plan)$  // Store plan in workspace
8:  $N_{steps} \leftarrow N_{steps} + 1$  // Count Planner execution as one step
9:
10: // Phase 2: Main Execution Loop
11:  $P_t \leftarrow \emptyset, F_t \leftarrow \emptyset$  // Initialize review decision and feedback
12: while  $N_{steps} < N_{\max}$  do
13:    $Plan \leftarrow \text{Workspace.ReadPlan}()$  // Load execution plan
14:    $T_{progress} \leftarrow \text{Workspace.GetCurrentState}()$  // Read current task progress
  
```

```

15:   $(S_t, R_t) \leftarrow \text{Manager}(Plan, T_{\text{Progress}}, R_{\text{Info}})$     // Manager selects next subtask and
                                                                    agent
16:   $N_{\text{steps}} \leftarrow N_{\text{steps}} + 1$                         // Count Manager decision as one step
17:  if  $S_t == \text{FINISH}$  or  $N_{\text{steps}} \geq N_{\text{max}}$  then
18:      break                                                // Stop if task finishes or step limit reached
19:  end if
20:   $(O_t, P_t, F_t, n_{\text{iter}}) \leftarrow \text{ExecuteAndReviewLoop}(S_t, R_t)$     // Execute subtask and
                                                                    perform review
21:   $N_{\text{steps}} \leftarrow N_{\text{steps}} + (2 \times n_{\text{iter}})$         // Count executor + reviewer per iteration
                                                                    as two steps
22:   $\text{Workspace.Update}(O_t)$                                 // Store execution output into workspace
23: end while
24:
25: // Phase 3: Termination
26:  $\mathcal{R}_{\text{final}} \leftarrow \text{Workspace.GetFinalResults}()$     // Retrieve final aggregated results
27: return  $\mathcal{R}_{\text{final}}$                                      // Return output to user

```

---

### 3.2. Tool calling

#### 3.2.1. Tool calling mechanism

In the GeoAgent framework, tools are fundamental agent components that provide the core capability for interacting with the external environment and are invoked by nodes during the agent workflow execution. To ensure consistent invocation across different tool types, each tool provides three key pieces of information to its node: a tool identifier for unique identification; a function description that explains the tool's purpose, applicable scenarios, and operational scope to the LLM in natural language; and a parameter specification that defines the required input structure, data type constraints, and output format. This standardized design can be formulated as:

$$\text{Tool}_i = \{\text{ID}_i, \text{DESC}_i, \text{SPEC}_i\}. \quad (8)$$

where  $\text{ID}_i$  is the unique tool identifier,  $\text{DESC}_i$  is the function description, and  $\text{SPEC}_i$  is the specification for input and output parameters.

The framework delegates the decision-making for tool selection and invocation to nodes equipped with reasoning capabilities. Each node is configured with a limited yet functionally essential set of tools. This design principle not only reduces the complexity of tool selection but also effectively lowers the probability of invocation errors by restricting the number of available tools. Furthermore, the node's role prompt contains explicit guidance on tool usage and execution steps, providing the LLM with contextual constraints. Consequently, tool selection and scheduling rely on the node's intrinsic LLM reasoning, the guidance from its role prompt, and the descriptive information provided by the tools themselves. This process varies by node type: a tool-augmented LLM node directly selects and invokes tools based on the current context and tool descriptions, whereas a ReAct Agent node follows the thought-action-observation loop to dynamically determine the timing of a tool call based on environmental feedback. The tool invocation process by a node can be represented as:

$$T_{\text{selected}} = \text{Node}(\text{Context}, \text{ToolSet}, \text{Prompt}). \quad (9)$$

where  $T_{\text{selected}}$  is the target tool selected by the node, *Context* denotes the current task context, *ToolSet* is the collection of tools accessible to the node, and *Prompt* is the node's role prompt.

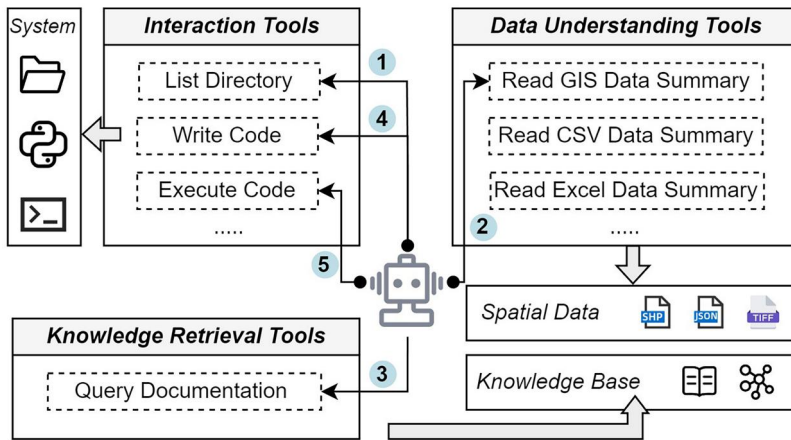
Once a node selects a tool, the actual execution is handled by the system's back-end Python environment. The system has a configured Python environment containing geospatial analysis libraries (e.g., GeoPandas, Rasterio, Shapely), ensuring that tools have access to the necessary computational resources and libraries. After the tool execution is complete, the tool returns a structured message to the node, containing details such as execution status, result data, and any exception information. This feedback enables the node to make subsequent decisions based on the tool's execution outcome.

### 3.2.2. Tool implementation

Recent autonomous GIS systems commonly provide LLMs with predefined sets of spatial analysis tools, while relatively fewer studies explore how LLMs might operate through more general-purpose and extensible tool interfaces. For complex spatial analysis workflows, however, organizing tool capabilities in a modular and decoupled manner is essential for building flexible and scalable analytical systems. To support this need, we abstracted the requirements for LLMs into three recurring functional dimensions: interaction with the computational environment, interpretation of heterogeneous geospatial data, and access to domain-specific analytical knowledge. Guided by this abstraction, we designed a taxonomy consisting of three complementary categories of tools: System Interaction Tools, Data Understanding Tools, and Domain Knowledge Retrieval Tools. Table 1 shows the tools designed in the framework under the three categories. Taking the Analyst as an example, Figure 6 shows the invocation sequence of the three tool types when it executes a task.

**Table 1.** Overview of the GeoAgent Toolset. To meet the diverse needs of agents in automated spatial analysis, the tools are systematically divided into three categories.

| Category                  | Tool Name                    | Tool Description                                        |
|---------------------------|------------------------------|---------------------------------------------------------|
| System Interaction Tools  | read_document                | Read document content.                                  |
|                           | read_plan_document           | Read execution plan documents generated by the Planner. |
|                           | write_document               | Write or modify document content.                       |
|                           | write_plan                   | Write execution plan documents.                         |
|                           | read_code                    | Read code scripts.                                      |
|                           | write_code                   | Write code files.                                       |
|                           | execute_code                 | Execute or modify code scripts.                         |
|                           | query_files_by_role          | Retrieve intermediate products by agent role.           |
|                           | list_working_directory_files | List intermediate products in the workspace.            |
|                           | save_file_metadata           | Save metadata of intermediate products for retrieval.   |
| Data Understanding Tools  | create_image_review_queue    | Create an image review queue for reviewers.             |
|                           | get_next_image_from_queue    | Fetch the next image entry from the review queue.       |
|                           | reading_csv_summary          | Read summaries of CSV files.                            |
|                           | reading_excel_summary        | Read summaries of Excel files.                          |
| Knowledge Retrieval Tools | reading_gis_metadata         | Read spatial metadata from GIS files.                   |
|                           | query_function_document      | Query specifications and usage of specific functions.   |



**Figure 6.** An illustrative workflow of the tool-calling process for an Analyst agent. The numbered arrows depict how the agent orchestrates interaction, data understanding, and knowledge retrieval tools to drive the analysis process from initial data understanding to final code execution.

System Interaction Tools play a key role in the GeoAgent framework by connecting LLMs with the external environment. They provide a standardized set of environmental operation interfaces, enabling LLMs to perform fundamental yet indispensable operations such as file system access, data reading and writing, and resource management.

Data Understanding Tools facilitate the agent's transition from passive generation to active decision-making. Geospatial data possess characteristics of scale dependence and topological complexity, which often necessitate an in-depth understanding of the data structure prior to analysis. To address this challenge, we designed Data Understanding Tools, endowing the agent with the capability for preliminary data comprehension. By automatically extracting key information such as spatial coverage, coordinate reference systems, attribute distribution characteristics, and spatial relationship patterns, the agent can construct a cognitive understanding of the data. This cognition enables agents to actively determine analysis strategies based on task requirements and data characteristics, including whether in-depth examination of specific datasets is needed, which analysis methods should be selected, and how key parameters should be set. Through this understand-then-generate analytical paradigm, agents can form systematic cognition of data characteristics before analysis, thereby formulating more targeted processing solutions.

Domain Knowledge Retrieval Tools provide the agent with the ability to keep pace with the latest technologies. Geographic information systems and their analytical methods are undergoing rapid iteration, with new algorithms and best practices constantly emerging, which puts pre-trained language models at risk of knowledge obsolescence. To counter this challenge, we designed knowledge retrieval tools using Retrieval-Augmented Generation technology.

A key implementation within this category is the function documentation query tool. This tool can invoke the built-in query interfaces of geospatial analysis libraries (e.g., GDAL, GeoPandas, Rasterio, Shapely) to retrieve critical technical information, such as function definitions and parameter specifications, by using a function's name. This method of retrieval, based on the library's documentation, ensures the

authoritativeness and accuracy of the information, while avoiding the time-sensitivity and instability problems of external web searches. When an agent needs to generate specific implementation code, this tool can accurately retrieve relevant domain knowledge and technical specifications, providing reliable technical support for code generation.

Through this architectural design with diverse tool collaboration, the GeoAgent framework achieves a systematic solution to the cognitive deficiencies of LLMs in the spatial analysis domain. In practical applications, these three types of tools do not form a simple linear calling relationship, but rather create a dynamically collaborative cognitive enhancement relationship. When faced with complex spatial analysis tasks, System Interaction Tools first ensure that the agent can acquire and manage the necessary data resources. Data Understanding Tools then initiate an in-depth analysis of data characteristics. Based on this understanding, agents can determine what types of professional knowledge are needed, and subsequently obtain relevant technical support through Domain Knowledge Retrieval Tools. Because the taxonomy is defined at a functional level rather than tied to specific implementations, it remains compatible with a wide range of future tool types. This design also provides a clear pathway for integrating cloud-based services, large-scale spatial databases, and specialized domain knowledge resources as the framework evolves.

## 4. Experiments

### 4.1. Setup

To systematically evaluate the performance and practicality of the GeoAgent framework, we constructed a representative dataset of analytical tasks. This dataset was primarily built upon spatial analysis tasks from ShapefileGPT and GIS Copilot, which provided representative cases for vector data processing and multi-data type processing, respectively (Lin *et al.* 2025, Akinboyewa *et al.* 2025).

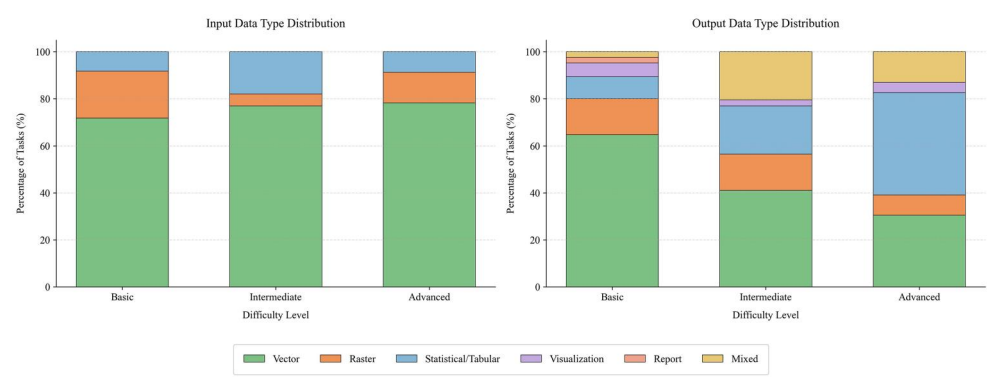
After filtering and integrating these cases, we finally constructed a comprehensive test set containing 147 spatial analysis tasks. To more finely measure the framework's ability to handle tasks of different complexity, we divided these tasks into three difficulty levels based on the length of the task's internal logical chain, the required operational steps, and the complexity of decisions, resulting in 85 basic tasks, 39 intermediate tasks, and 23 advanced tasks. Table 2 shows typical task examples for each difficulty level. The criteria for dividing the difficulty levels are as follows:

- Basic tasks refer to operations that can be completed in a single step or by invoking a single core tool. These tasks have clear objectives and short logical chains.
- Intermediate tasks require a linear workflow of 2–4 steps and typically involve interactions between multiple data layers or a sequence of geoprocessing operations.
- Advanced tasks involve complex planning, multi-source data fusion, or multi-criteria decision making. These comprehensive problems typically require multiple types of outputs.

Figure 7 illustrates the distribution of input and output file types across different difficulty levels. The input data types consist of vector data, raster data, and tabular

**Table 2.** Definitions and representative examples for the three task difficulty levels. The classification is based on workflow complexity.

| Difficulty Level   | Prompt Example                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Basic Tasks        | <b>Example 1:</b> Separate each band of the input image into individual TIF files.<br><b>Example 2:</b> Clip the digital elevation model to the boundaries of Pennsylvania and save the result as a new raster file.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Intermediate Tasks | <b>Example 1:</b> Perform the following sequence of tasks: (1) Count the number of fast-food restaurants in each county and store the result in a new field named Count. (2) Calculate the fast-food accessibility score for each county as $(\text{Count}/\text{Population}) \times 1,000$ and store the result in a new field named Score. (3) Create a thematic map visualizing the fast-food accessibility score for each county.<br><b>Example 2:</b> Add new fields for latitude and longitude to the fast food restaurants shapefile, then extract the elevation values from the DEM raster and save the results in a new shapefile.                                                                                                                                                    |
| Advanced Tasks     | <b>Example 1:</b> Identify counties in Pennsylvania that are suitable for tree planting based on annual rainfall. Counties receiving more than 2.5 inches of rain per year should be classified as suitable. Report the percentage of the state that meets this condition and the number of suitable counties.<br><b>Example 2:</b> Within the study area, determine a proposed location for a new Walmart store that is approximately equidistant from the four existing store locations, with a maximum distance deviation of 2,000 meters. The new location must be within 300 meters of a road segment. Save the proposed location as a point feature in a new shapefile, and generate a map that plots the four existing stores and the new site, including a basemap and labeled points. |



**Figure 7.** Distribution of input and output data types across task difficulty levels (Basic, Intermediate, Advanced) in the experimental dataset. Input types primarily consist of Vector, Raster, and Statistical/Tabular data. Output types show greater diversity, particularly for intermediate and advanced tasks, including Visualization, Report, and Mixed formats.

data. The distribution of output data types is more diverse, particularly in intermediate and advanced tasks, where there is a larger proportion of visualization, report, and mixed-type outputs. This reflects the demand of complex spatial analysis tasks for diverse output formats, especially an increased need for data visualization and result interpretation.

For performance evaluation, we used the task success rate as the primary metric. A task is considered successful if and only if all of its required outputs are generated correctly and meet the task's specifications. Depending on the specific objectives, these outputs may include one or more of the following: geospatial data files must be in a valid format with correct information; visualizations must reflect the analysis results and adhere to specified norms; and analytical reports must present reliable

conclusions and include all required content. A detailed checklist is provided in Table B1, which specifies the criteria used to determine whether each output is correct and satisfies the task requirements, thereby facilitating consistent human evaluation during manual evaluation. Given the stochastic nature of LLM outputs, we employed a Pass@2 standard to calculate the success rate, thereby mitigating the impact of random failures in a single run (Chen *et al.* 2021). Specifically, each task was run twice in completely independent end-to-end trials. The task was marked as successful if at least one of these two trials succeeded.

To test the framework's comprehensive capabilities on complex, real-world problems, we selected representative tasks from the Chinese College Students GIS Contest to serve as the case study in Section 4.5. These tasks are not part of the test dataset but are classified as advanced tasks.

#### 4.2. Performance evaluation

To comprehensively evaluate the performance of GeoAgent, we designed a comparative study against three baseline systems. We included two single-agent baselines: GPT-4o, configured with its native code interpreter, and GPT-4o (Adapted). The adapted version resolves incompatibility issues among geospatial analysis dependency libraries within the interpreter, providing a stable execution environment that reduces configuration-related errors. Both single-agent systems possess end-to-end capabilities for reasoning, coding, and execution, with their primary difference being the degree of adaptation to the geospatial environment. The multi-agent baseline, GeoJsonAgent, is built on a planner-executor paradigm driven by GPT-4o, where a planning module generates a task decomposition scheme and an executor module performs the resulting geospatial operations.

We designed two model configuration schemes to evaluate the framework's performance. The first configuration employs OpenAI's GPT series models, using GPT-4o as the core for the Planning Layer and Manager agent, while other agents in the Execution Layer and Review Layer utilize GPT-4o-mini. The second configuration is primarily based on the DeepSeek-V3 model; however, since DeepSeek-V3 currently does not support multimodal processing, we retained GPT-4o-mini only for the image review component. We selected both closed-source and open-source models for comparative experiments to assess the framework's adaptability and universality under different types of large model support.

Table 3 shows the detailed experimental result comparison. In terms of overall performance, the GeoAgent framework demonstrated significant improvements compared

**Table 3.** Performance comparison of GeoAgent and baseline methods on the 147-task test set.

| Method                                           | Basic (n = 85) |      |         | Intermediate (n = 39) |      |        | Advanced (n = 23) |      |        | Total (n = 147) |      |        |
|--------------------------------------------------|----------------|------|---------|-----------------------|------|--------|-------------------|------|--------|-----------------|------|--------|
|                                                  | Succ.          | Fail | Rate    | Succ.                 | Fail | Rate   | Succ.             | Fail | Rate   | Succ.           | Fail | Rate   |
| GPT-4o                                           | 51             | 34   | 60.00%  | 20                    | 19   | 51.28% | 8                 | 15   | 34.78% | 79              | 68   | 53.74% |
| GPT-4o (Adapted)                                 | 65             | 20   | 76.47%  | 28                    | 11   | 71.79% | 11                | 12   | 47.83% | 104             | 43   | 70.75% |
| GeoJsonAgent(GPT-4o)<br>(Luo <i>et al.</i> 2026) | 76             | 9    | 89.41%  | 30                    | 9    | 76.92% | 14                | 9    | 60.87% | 120             | 27   | 81.63% |
| GeoAgent (GPT-4o)                                | 85             | 0    | 100.00% | 34                    | 5    | 87.18% | 20                | 3    | 86.96% | 139             | 8    | 94.56% |
| GeoAgent (DeepSeek-V3)                           | 81             | 4    | 95.29%  | 37                    | 2    | 94.87% | 22                | 1    | 95.65% | 140             | 7    | 95.24% |

to the baseline systems. The single-agent baseline GPT-4o (Adapted) successfully completed 104 of 147 test cases, for an overall success rate of 70.75%. The multi-agent baseline GeoJsonAgent showed improved performance, reaching a success rate of 81.63%, an improvement of 10.88% over the single-agent baseline, highlighting the advantages of multi-agent collaboration. However, our proposed GeoAgent framework exhibited superior performance under both configurations: the GPT-4o-based GeoAgent configuration achieved a 94.56% success rate, while the DeepSeek-V3-based configuration reached 95.24%, which are 23.81% and 24.49% higher than the single-agent baseline GPT-4o (Adapted), and 12.93% and 13.61% higher than the multi-agent baseline, respectively.

In terms of performance on tasks of different difficulty levels, the framework's advantages were evident across all complexity levels. For basic tasks, the success rate of the single-agent baseline GPT-4o (Adapted) was 76.47%, the multi-agent baseline GeoJsonAgent reached 89.41%, while the two GeoAgent configurations achieved 100.00% and 95.29% success rates, respectively, showing the reliability of the hierarchical architecture in handling basic spatial analysis operations. The performance gap becomes more pronounced for intermediate tasks. The single-agent baseline only reached a 71.79% success rate, the multi-agent baseline was 76.92%, while GeoAgent's two configurations achieved 87.18% and 94.87% success rates, with the DeepSeek-V3 configuration being particularly outstanding. In the most challenging advanced tasks, the limitations of the baseline systems became more apparent, with the single-agent baseline's success rate dropping to 47.83% and the multi-agent baseline at 60.87%, while the GeoAgent framework still maintained a high level of performance, with the GPT-4o configuration reaching 86.96% and the DeepSeek-V3 configuration reaching 95.65%.

As task complexity increased, the success rate of most methods declined, reflecting the greater demands that advanced spatial analyses place on reasoning and the application of domain knowledge. It is noteworthy that while the two GeoAgent configurations have similar overall success rates, they exhibit different strengths. The GPT-4o configuration achieved a perfect score on basic tasks, showing its high reliability in handling well-defined, simple spatial analysis operations; while the DeepSeek-V3 configuration excelled on intermediate and advanced tasks (94.87% and 95.65%), indicating its stronger adaptability in spatial analysis tasks requiring multi-step planning and complex reasoning. Despite the challenges posed by increasing complexity, both GeoAgent configurations maintained success rates of at 86.96% even on the most difficult advanced tasks. This demonstrates the robustness and efficacy of GeoAgent's hierarchical architecture for tackling complex spatial analysis problems.

#### ***4.3. Efficiency and cost analysis***

While multi-agent systems enhance complex problem-solving capabilities, their inherent communication and coordination overhead makes operational efficiency and economic cost critical dimensions for evaluation. We introduced three key efficiency metrics and recorded the performance of GeoAgent across tasks of varying difficulty. The analysis in this section is based on the GeoAgent (GPT-4o) model configuration

**Table 4.** Average efficiency and cost metrics per agent for the GeoAgent. The table details the token usage, latency, and estimated cost for each agent across the three task difficulty levels.

| Agent                  | Basic  |             |           | Intermediate |             |           | Advanced |             |           |
|------------------------|--------|-------------|-----------|--------------|-------------|-----------|----------|-------------|-----------|
|                        | Tokens | Latency (s) | Cost (\$) | Tokens       | Latency (s) | Cost (\$) | Tokens   | Latency (s) | Cost (\$) |
| <b>Planning Layer</b>  |        |             |           |              |             |           |          |             |           |
| Planner                | 11750  | 33.5        | 0.0251    | 14566        | 30.9        | 0.0326    | 14310    | 30.9        | 0.0320    |
| <b>Execution Layer</b> |        |             |           |              |             |           |          |             |           |
| Manager                | 7071   | 7.5         | 0.0186    | 9639         | 9.0         | 0.0247    | 9873     | 8.9         | 0.0251    |
| Analyst                | 43648  | 88.8        | 0.0063    | 72322        | 153.9       | 0.0101    | 65417    | 130.5       | 0.0093    |
| Cartographer           | 38007  | 54.6        | 0.0052    | 64149        | 112.8       | 0.0084    | 48336    | 88.7        | 0.0071    |
| Reporter               | 14262  | 34.0        | 0.0021    | –            | –           | –         | –        | –           | –         |
| <b>Review Layer</b>    |        |             |           |              |             |           |          |             |           |
| Analysis Reviewer      | 1332   | 1.8         | 0.0002    | 1557         | 2.3         | 0.0003    | 1606     | 1.9         | 0.0003    |
| Image Reviewer         | 32442  | 10.9        | 0.0048    | 30995        | 12.6        | 0.0046    | 20100    | 9.7         | 0.0030    |
| Report Reviewer        | 2509   | 3.4         | 0.0004    | –            | –           | –         | –        | –           | –         |

from Table 3, in which the Planner and Manager agents are powered by GPT-4o, while all agents in the Execution and Review Layers use GPT-4o-mini.

The three efficiency metrics we introduced are defined as follows: The Tokens metric measures the sum of prompt and completion tokens consumed by an agent in a single task. The Latency metric measures the wall-clock time consumed by a single agent to perform all its operations, reflecting the actual time cost of each agent. The Cost metric is estimated based on the model provider’s per-token pricing. Table 4 shows the average efficiency metrics for each agent in the framework during a single task execution.

The experimental results reveal clear resource consumption distribution characteristics within the framework. Among the three layers, the Execution Layer constitutes the main body of resource consumption, which is consistent with our design of positioning it as the core task-processing unit. Specifically, the Analyst and Cartographer are the two agents with the highest resource consumption. For example, when processing intermediate tasks, the Analyst’s average token consumption was 72,322 and latency was 153.9 seconds, while the Cartographer’s average token consumption was 64,149 and latency was 112.8 seconds. This phenomenon can be attributed to their hybrid node architecture, which involves multiple rounds of tool calls and iterative optimization by the ReAct agent mechanism, leading to higher computational overhead. In contrast, the resource costs of the Planning and Review Layers were significantly lower. The Planner’s token consumption and latency remained at a low and stable level across different difficulty tasks, indicating that the overhead of its task decomposition stage is controllable. The agents in the Review Layer generally have lower resource consumption because their tasks focus on verifying existing results rather than generating new ones. Among them, the Image Reviewer has a relatively higher token consumption due to the need to process visual inputs.

From the perspective of task difficulty, the average resource consumption of intermediate tasks was higher than that of advanced tasks. We infer that this phenomenon stems from the inherent structure of some intermediate tasks. Although the complexity of individual steps is lower, they often contain longer and more numerous data processing steps. In contrast, while advanced tasks are logically more complex, their

core analysis process may be more condensed and can be completed in fewer steps. The difficulty of advanced tasks stems not from the number of steps but from the more complex reasoning required for each individual step, even if the overall workflow is shorter. Therefore, the cumulative interaction steps of the former led to higher token consumption and latency.

In our test dataset, the intermediate and advanced tasks did not include requirements for report generation. Consequently, the Reporter agent was not activated for these difficulties, and its corresponding entries in Table 4 are empty. This design highlights a resource-aware allocation feature of the framework. Agents are invoked only when a task explicitly requires their specific type of output, thereby conserving resources.

Overall, the efficiency analysis validates the rationality of the GeoAgent framework's resource utilization. The Execution Layer bears the primary computational load, consistent with its core functional role. The Planning and Review Layers have low overhead but provide significant value by ensuring accurate task decomposition and quality assurance, respectively. This resource allocation method offers a valuable reference for designing more efficient intelligent spatial analysis systems.

#### **4.4. Ablation experiment**

To reveal the effectiveness and necessity of each hierarchical component in the GeoAgent framework, we designed systematic ablation experiments. As the Execution Layer constitutes the minimal functional unit for GeoAgent to operate, we designed three variant versions of the GeoAgent framework: (1) a version without the Planning and Review Layers (GeoAgent Execution Only), representing the most basic processing mode; (2) a version without the Planning Layer (GeoAgent w/o Planning), where the Execution Layer directly receives and executes user requirements; (3) a version without the Review Layer (GeoAgent w/o Review), which removes the result review and optimization feedback loops. Considering the potential impact of model choice on framework performance, we conducted the ablation experiments under two different LLM configurations, GPT-4o and DeepSeek-V3, with the model setups being consistent with Section 4.2.

By comparing the performance differences between the complete framework and simplified versions with specific layers removed on the same task set, we can evaluate the contribution of each component and gain a deeper understanding of the value of multi-agent collaborative mechanisms in spatial analysis. Specific results can be seen in Table 5.

##### **4.4.1. Individual layer ablation analysis**

**4.4.1.1. Contribution analysis of the planning layer.** As the primary component of the GeoAgent framework, the Planning Layer is responsible for task parsing and execution planning. The variant without the Planning Layer directly passes user task descriptions and spatial data to the Execution Layer for processing, skipping preliminary steps such as task complexity assessment, category recognition, data type determination, and subtask decomposition.

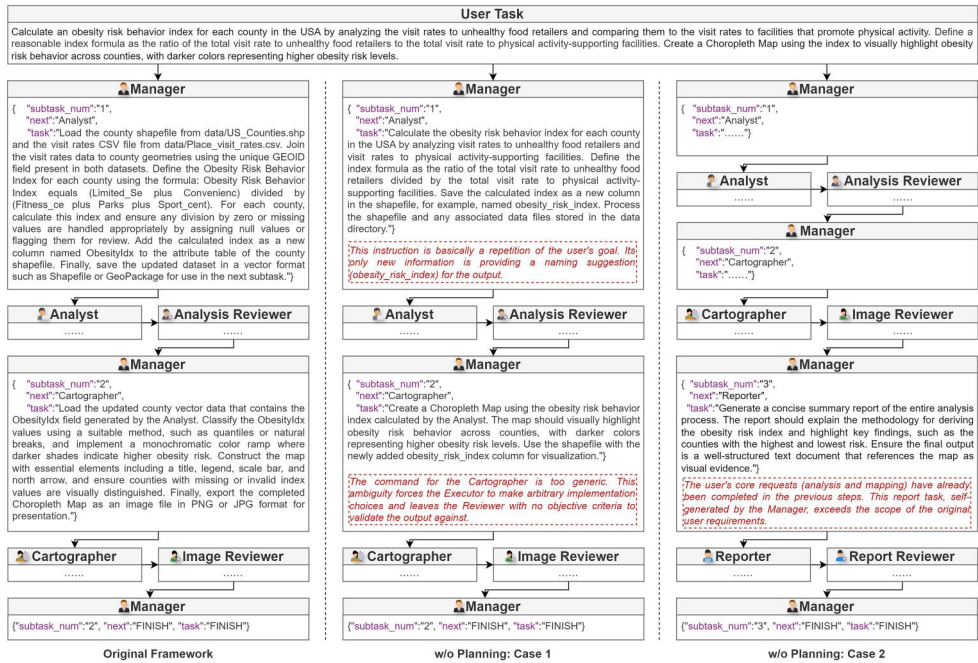
**Table 5.** Ablation study results for the framework. The experiment, under both GPT-4o and DeepSeek-V3 model configurations, evaluated the contribution of the Planning and Review Layers to the overall task success rate.

| Difficulty Level | Total Cases | Original Framework |      |         | Execution Only |      |        | w/o Planning Layer |      |        | w/o Review Layer |      |        |
|------------------|-------------|--------------------|------|---------|----------------|------|--------|--------------------|------|--------|------------------|------|--------|
|                  |             | Success            | Fail | Rate    | Success        | Fail | Rate   | Success            | Fail | Rate   | Success          | Fail | Rate   |
| GPT-4o           |             |                    |      |         |                |      |        |                    |      |        |                  |      |        |
| Basic            | 85          | 85                 | 0    | 100.00% | 59             | 26   | 69.41% | 63                 | 22   | 74.12% | 68               | 17   | 80.00% |
| Intermediate     | 39          | 34                 | 5    | 87.18%  | 20             | 19   | 51.28% | 17                 | 22   | 43.59% | 20               | 19   | 51.28% |
| Advanced         | 23          | 20                 | 3    | 86.96%  | 7              | 16   | 30.43% | 7                  | 16   | 30.43% | 14               | 9    | 60.87% |
| Total            | 147         | 139                | 8    | 94.56%  | 86             | 61   | 58.50% | 87                 | 60   | 59.18% | 102              | 45   | 69.39% |
| DeepSeek-V3      |             |                    |      |         |                |      |        |                    |      |        |                  |      |        |
| Basic            | 85          | 81                 | 4    | 95.29%  | 63             | 22   | 74.12% | 66                 | 19   | 77.65% | 64               | 21   | 75.29% |
| Intermediate     | 39          | 37                 | 2    | 94.87%  | 21             | 18   | 53.85% | 26                 | 13   | 66.67% | 22               | 17   | 56.41% |
| Advanced         | 23          | 22                 | 1    | 95.65%  | 10             | 13   | 43.48% | 16                 | 7    | 69.57% | 13               | 10   | 56.52% |
| Total            | 147         | 140                | 7    | 95.24%  | 94             | 53   | 63.95% | 108                | 39   | 73.47% | 99               | 48   | 67.35% |

The experimental results under the GPT-4o configuration reveal that after removing the Planning Layer, the overall success rate of the framework significantly decreased from 94.56% to 59.18%, a substantial decline of 35.38 percentage points. Compared to the Execution-only variant, the w/o Review Layer variant which includes the Planning Layer performed significantly better, an increase of 10.89 percentage points. This significant difference strongly demonstrates the irreplaceable role of the Planning Layer in automating spatial analysis tasks. In-depth analysis of performance across datasets of different complexities reveals that the importance of the Planning Layer becomes increasingly prominent as task complexity increases. In the set of basic tasks, removing the Planning Layer caused the success rate to drop from 100.00% to 74.12%; while in intermediate and advanced task sets, success rates plummeted from 87.18% and 86.96% to 43.59% and 30.43% respectively. This widening performance gap with increasing complexity clearly indicates that the hierarchical task parsing and planning mechanism of the Planning Layer plays a crucial role in processing complex spatial analysis tasks.

We found that without the Planning Layer, the Manager agent in the Execution Layer is forced to simultaneously assume dual responsibilities of task parsing and execution planning, significantly increasing its cognitive burden and leading to markedly decreased performance. This was specifically manifested in three main aspects: First, task termination judgment was imprecise; even among the 87 tasks deemed successful, 15 included correct results but also contained redundant processing outputs, leading to wasted computational resources and reduced efficiency. Second, the error accumulation effect was obvious; initial misinterpretations of user instructions by the Manager led to subsequent task processing continuously deviating from the correct path, creating an error amplification phenomenon. Finally, subtask decomposition was either missing or irrational, demonstrated by the Manager either passing the entire complex instruction directly to an execution agent or performing illogical task splitting, which resulted in chaotic execution flows or the omission of critical steps.

Figure 8 illustrates two of these failure modes. As shown in case 1, the Manager might pass a directive downstream that is nearly identical to the user's high-level goal. Alternatively, as shown in case 2, it might generate instructions that go beyond the scope of the original request.

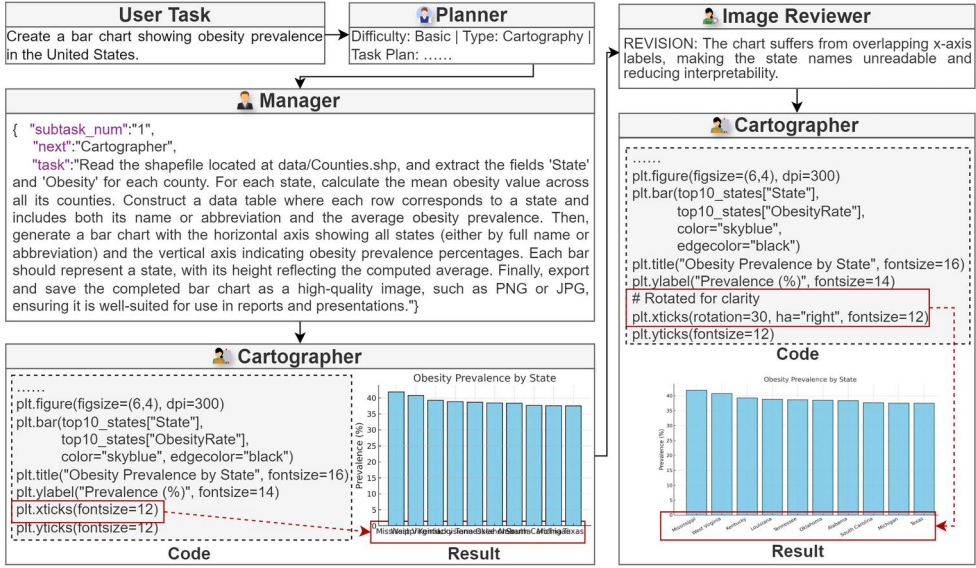


**Figure 8.** A comparison of three workflows illustrates the necessity of the Planning Layer. The original framework (left) demonstrates a successful run, in which the user's request is correctly decomposed into specific analysis and cartography subtasks. In the first failure scenario (middle), the Manager provides the Cartographer with a vague and under-specified instruction, which compromises the quality of the final map and leaves the reviewer without objective criteria for validation. The second failure scenario (right) shows the Manager generating a redundant, out-of-scope subtask after the user's core requirements have been met.

**4.4.1.2. Contribution analysis of the review layer.** The Review Layer serves as a quality assurance mechanism for the GeoAgent framework, responsible for professional evaluation and feedback optimization of execution results. The variant without the Review Layer directly outputs results generated by the Execution Layer, lacking the iterative improvement process of the Review Layer.

After removing the Review Layer, the framework's success rate on the test set under the GPT-4o configuration dropped from 94.56% to 69.39%, a decrease of 25.17 percentage points. Compared to the Execution-Only variant, the w/o Planning Layer variant, which includes the Review Layer, performed better with a success rate of 59.18%. Further analyzing the performance differences on tasks of different complexities, we find that the role of the Review Layer also shows a characteristic of becoming more important as task complexity increases.

Through a systematic analysis of failure cases, we found that the primary issues with the variant lacking the Review Layer were concentrated in the accuracy and reliability of the results. Specific manifestations include: First, the system's error recovery capability significantly declined. When the Execution Layer encountered unexpected situations during processing (such as data anomalies or code execution errors), the Manager was forced to undertake the responsibility of correcting the current task and



**Figure 9.** Demonstration of the self-verification and iterative optimization mechanism enabled by the Review Layer. The process unfolds in a feedback loop: (1) The Cartographer agent's initial attempt produces a bar chart (bottom left) with a significant visual flaw—overlapping, unreadable x-axis labels. (2) The Image Reviewer automatically identifies this defect and provides a clear, actionable revision instruction. (3) The Cartographer incorporates the feedback, modifies its code to rotate the x-axis labels, and successfully generates a corrected, high-quality visualization (bottom right).

accurately re-dispatching the failed subtask. However, we observed that the expression details of subtask instructions were often lost during multiple inter-agent communications, consequently affecting accurate task execution. This error accumulation effect, due to the lack of professional review feedback, was particularly pronounced in complex tasks. Second, cartography-related tasks often failed to meet professional standards and user requirements. This was primarily because the system lost the critical stage of conducting professional visual validation and cartographic standard reviews for the output results. Even if the analysis process itself contained no logical errors, the final visual products frequently suffered from the absence of professional calibration based on cartographic principles. In particular, they often failed to effectively avoid and correct common visual expression issues such as symbol stacking and legend conflicts, leading to obscured communication of spatial information, a lack of emphasis on key points, and even potential visual misinterpretation. Figure 9 shows an intuitive example of this situation.

#### 4.4.2. Cross-model configuration robustness analysis

To validate the generalization performance and architectural robustness of the GeoAgent framework design, we repeated the ablation study under two different LLM configurations, GPT-4o and DeepSeek-V3.

Comparing the experimental results of the two models in Table 5, it can be found that different model configurations have different degrees of dependency

on different layers. In the GPT-4o configuration, removing the Planning Layer (59.18%) resulted in a greater performance loss than removing the Review Layer (69.39%); whereas in the DeepSeek-V3 configuration, the situation was the opposite; the performance loss from removing the Review Layer (67.35%) exceeded that of removing the Planning Layer (73.47%). A possible explanation for this difference is the different intrinsic capabilities of the models used in the Execution Layer in the two configurations. In the GPT-4o configuration, the model driving the Execution Layer is GPT-4o-mini, which, as a more cost-effective model, has relatively limited ability to independently handle complex tasks and may rely more on the task decomposition and detailed execution guidance provided by the Planning Layer. In the DeepSeek-V3 configuration, the Execution Layer uses the DeepSeek-V3 model, which exhibits strong reasoning and problem-solving abilities, and can maintain a relatively high task execution performance even in the absence of external planning guidance.

Although the two model configurations have differences in their specific layer dependency patterns, the complete framework is always superior to all simplified variants, and each hierarchical component has demonstrated its value. This consistency of architectural advantage across models indicates that the synergistic value of the components in the framework has relatively stable characteristics and is not entirely dependent on the individual capabilities of a specific LLM, which provides an architectural paradigm reference for the design of geospatial intelligent systems.

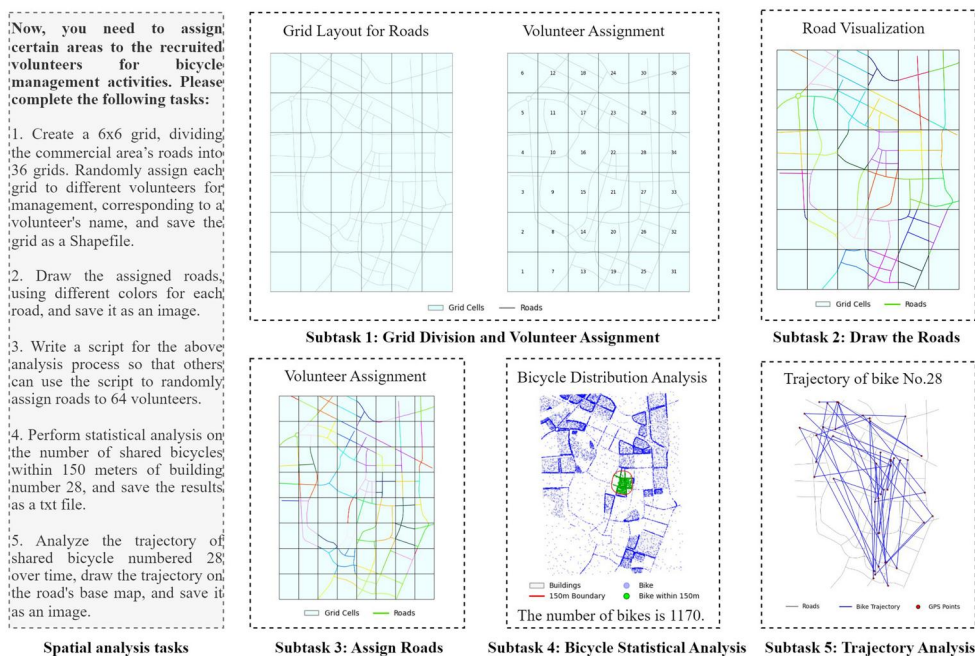
#### **4.5. Case study**

This section systematically evaluates the capabilities and effectiveness of the GeoAgent multi-agent framework in solving practical spatial analysis problems through two carefully designed cases. The instructions for these cases, featuring clear objectives and presented as a series of explicit substeps, are intended to verify the framework's flexibility, analytical precision, and practical value when applied to complex spatial analysis tasks.

##### **4.5.1. Case 1: spatial resource optimization**

The first case focuses on urban shared bicycle management, involving multidimensional spatial analysis tasks such as spatial grid division, dynamic resource allocation, and trajectory analysis. As shown in [Figure 10](#), the GeoAgent framework first divided the commercial district road network into a grid system and implemented random volunteer allocation, subsequently generating a color-differentiated road responsibility area map, while also developing an automation script to support larger-scale resource allocation. Within building perimeter areas, the framework completed bicycle count statistical analysis, ultimately achieving trajectory reconstruction and visualization for specific bicycles.

The coherent execution of this series of spatial analysis tasks fully demonstrated that the GeoAgent framework's Planning Layer can effectively decompose complex tasks, the Execution Layer can accurately complete various spatial operations, and the Review Layer can ensure the accuracy and consistency of the results. The multi-agent



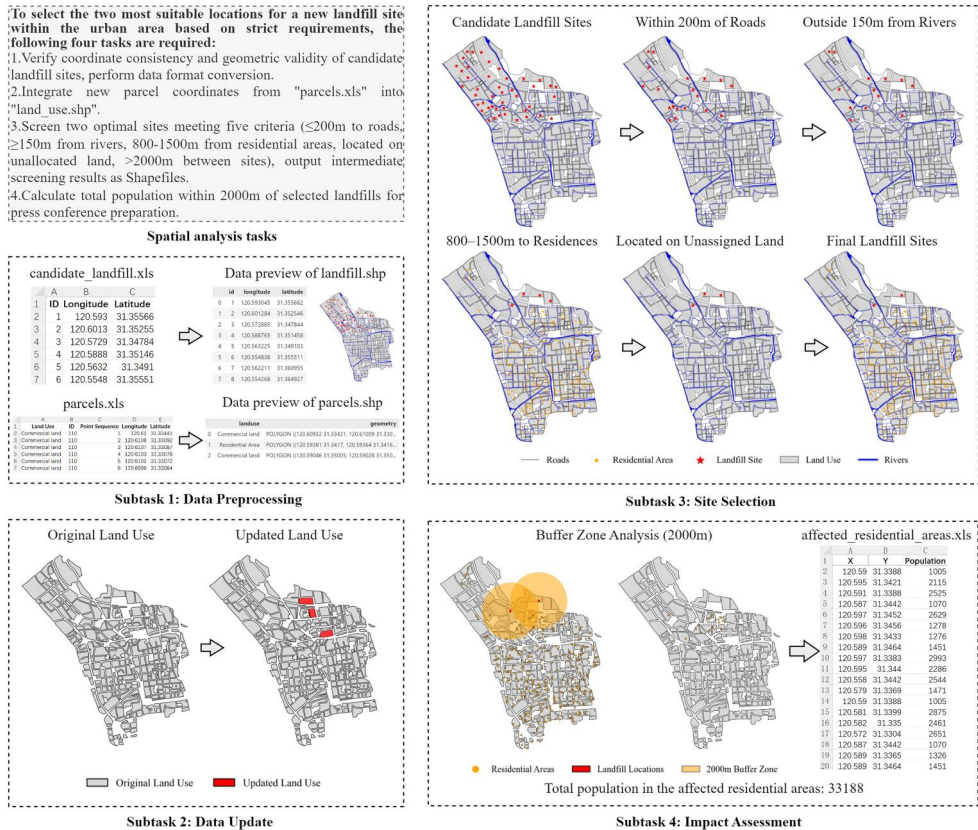
**Figure 10.** The case study of an urban bicycle management scenario. The automated workflow includes grid-based road division, volunteer assignment, road network visualization, bicycle distribution analysis, and individual trajectory tracking.

collaborative mechanism showcased significant advantages in integrating complex spatial analysis workflows, achieving a complete automated workflow from data input to result output. This validates the framework's practical value and applicability in the domain of urban spatial management and resource optimization allocation.

#### 4.5.2. Case 2: multi-criteria site selection

The second case study explored the urban facility site selection problem under multi-criteria spatial constraints, representing a typical complex spatial decision-making challenge. Through this case, we primarily investigated the GeoAgent framework's capabilities in handling multi-source heterogeneous data integration, multi-criteria spatial constraint evaluation, and multi-criteria decision support.

As shown in Figure 11, the framework, through its hierarchical agent structure, decomposed the complex site selection task into four coherent subtasks: data preprocessing, land use data updating, multi-criteria site selection analysis, and impact assessment. In the data preprocessing stage, the framework verified coordinate consistency and geometric validity of candidate landfill locations. Subsequently, the system integrated field-collected parcel information to update the land use data layer. During the multi-criteria site selection stage, the framework, based on five spatial constraint conditions—road accessibility, water body protection distance, residential area impact zone, land use type, and inter-facility spacing—determined the optimal landfill location through progressive filtering. Finally, the system calculated the total



**Figure 11.** The case study of a multi-criteria urban landfill site selection problem. GeoAgent completed subtasks including data preprocessing and validation, land use data updating, progressive multi-criteria spatial filtering for optimal site identification, and population impact assessment.

population within the impact area around the landfill to provide a scientific basis for decision-making.

This entire case fully demonstrated the GeoAgent framework's advantages in spatial decision support. It not only achieved end-to-end automation from data processing to solution evaluation but also preserved intermediate analysis results at each stage, making the decision-making process more transparent. Through the validation of this case, the framework exhibited significant application value and solution integration capability in complex decision-making problems that require the simultaneous consideration of multiple spatial constraints.

## 5. Discussion

### 5.1. Insights from main experiments

Both framework configurations based on GPT-4o and DeepSeek-V3 demonstrated high overall success rates, with the DeepSeek model performing slightly better than the GPT model in intermediate and advanced tasks. This indicates that our proposed hierarchical multi-agent framework possesses a certain degree of cross-model generality.

Although system success rates declined as task complexity increased, the decline was relatively manageable for GeoAgent, whereas the single-agent baseline exhibited a much steeper decrease in performance. Given the characteristics of the evaluation dataset and the strict evaluation method adopted in this study, single-agent systems face inherent challenges under this experimental setting. A single agent must simultaneously handle task interpretation, execution, and outcome verification within a single continuous reasoning trajectory, making it more susceptible to cumulative errors as task depth increases. Typical failure cases include prematurely assuming task completion merely because the generated code executes without errors—even when the resulting outputs are empty, incomplete, or geometrically incorrect—or proceeding directly with downstream analysis without performing necessary data quality checks and preprocessing when input data contain inconsistencies or missing information.

In contrast, the hierarchical structure of the framework mitigates these issues through explicit role separation and structured workflow coordination. The Planning Layer is able to identify data characteristics and incorporate required preprocessing steps into the execution plan, ensuring that potential inconsistencies are resolved before analysis begins. The Review Layer evaluates intermediate and final outputs from an independent perspective, focusing not only on whether the code runs but also on whether the produced artifacts are correct and meet the task requirements. This division of responsibilities reduces the propagation of local errors across execution stages and prevents the premature acceptance of incomplete or incorrect results. As a consequence, GeoAgent exhibits higher robustness and reliability on complex, multi-stage spatial analysis tasks compared with the single-agent baseline.

## **5.2. Implications from ablation experiments**

The ablation experiment results reveal the differentiated value of the Planning Layer and Review Layer across tasks of varying complexity. For simple spatial analysis tasks (such as overlay analysis or buffer analysis), the contributions of these two layers are relatively limited; for complex tasks (such as site selection or suitability analysis), they are key components ensuring task completion and result reliability. As task complexity increases, the Planning Layer's role in task decomposition and process design becomes more critical, while the value of the Review Layer's optimization suggestions also substantially increases. This phenomenon indicates that when task complexity exceeds a certain degree, the value of professional division of labor increases dramatically.

The Planning Layer primarily underpins the logical integrity of the workflow, ensuring that subtask division is rational, the execution sequence is valid, and task allocation is appropriate. By pre-defining clear subtask boundaries and logical dependencies, the Planning Layer generates the essential context that guides the Manager agent in orchestrating the execution process. Without this structured guidance, the system risks losing logical coherence in multi-stage workflows, leading to potential mismanagement of task sequences.

The Review Layer enhances the quality and compliance of diverse analytical products. Spatial analysis tasks typically require communicating insights to decision-makers through a variety of output formats, such as vector or raster datasets, cartographic visualizations, statistical charts, and textual reports. These artifacts must not only be technically correct but also adhere to professional domain standards. We introduce an independent Review

Layer to address this requirement, thereby improving the system's capability to deliver accurate, complete, and professionally readable results in complex tasks.

### **5.3. Error mechanism analysis**

Error control and propagation are critical factors that affect the reliability of multi-agent systems. In this study, for error control within individual agents, the framework provides a deterministic information source for each agent through the tool calling mechanism, avoiding hallucinatory errors based on speculation. At the same time, the framework constrains agent behavior through a pre-defined workflow, and designs the context for each node, ensuring that each node is responsible only for a specific, focused task. Regarding the problem of error propagation between agents, the hierarchical architecture adopted in this study reflects the design feature of a directed information flow. Information mainly flows along the one-way path of planning, execution, and review, with a feedback mechanism existing only between the execution and review stages. Between different layers, information transfer is mainly achieved through objective products like execution plans and data files, which can isolate the contextual information between different agents.

However, in the current design, a misunderstanding by the Planner or Manager agent can affect the entire execution workflow, and other agents in the system do not undertake the responsibility of error detection across functional modules. Building a reliable autonomous spatial analysis system may require finding a pragmatic balance between raising the upper limit of professional capabilities and maintaining system stability. It is necessary to achieve efficient collaboration through a clear division of labor, while also maintaining sufficient robustness to cope with various uncertain challenges in real-world applications.

### **5.4. Evaluation scope and future directions**

The evaluation design of this study focuses on assessing GeoAgent's ability to complete spatial analysis tasks in an end-to-end manner. In particular, it examines whether the system can independently interpret natural-language instructions, perform reasonable task decomposition, invoke appropriate geospatial tools, and ultimately produce analysis results that meet professional standards. This task-level evaluation strategy aligns closely with the core objectives of the framework and provides a solid basis for verifying the stability and overall reliability of a hierarchical multi-agent system operating in real geospatial computing environments. Since spatial analysis outputs often need to satisfy multiple requirements, including geometric accuracy, spatial-logical validity, attribute correctness, and cartographic expression, expert review is still necessary. Such review ensures the professional soundness and accuracy of complex spatial products, which current automated procedures are not yet able to fully guarantee.

A broader view of existing geospatial LLM and agent-based systems provides additional context for interpreting the evaluation scope adopted in this study. [Table B2](#) offers a summary of representative frameworks, outlining differences in system design, task focus, and commonly used evaluation practices. These variations reflect the diverse objectives pursued across current research rather than a unified assessment paradigm, and the task-level evaluation used here should therefore be understood as one reasonable choice aligned with the operational characteristics of GeoAgent.

Future improvements to the evaluation framework may proceed along two dimensions. One direction is the development of more fine-grained and automated assessment methods. At the metric level, process-oriented indicators—such as quantitative measures of plan coherence or execution stability—could capture aspects of multi-agent coordination that are not reflected in task-level success rates. Recent benchmarks such as GeoAnalystBench (Zhang *et al.* 2025) have demonstrated the value of evaluating models through analyses of code-generation logic or workflow structure. Although their objectives differ from this study's emphasis on end-to-end task completion, they highlight the importance of process-oriented assessment and offer useful reference points for developing more granular evaluation frameworks in future work. Additional result-level metrics, including spatial accuracy or the consistency of cartographic outputs, would further enrich the characterization of final outputs. Although the evaluation dataset constructed in this research has covered various typical spatial analysis task categories, the coverage of tasks remains limited in areas such as advanced spatial statistics, complex spatiotemporal pattern mining, and complex scenarios involving real-time data processing, dynamic environments, or large-scale data integration. Therefore, in future work, incorporating more advanced or interdisciplinary spatial analysis tasks, particularly those intersecting with emerging fields such as urban intelligence, climate change analysis, and ecosystem assessment, could further enrich the breadth and depth of the evaluation dataset.

### **5.5. Future prospects for the system framework**

This research clearly reveals the value of hierarchical agent collaboration in enhancing spatial analysis. By decomposing complex spatial analysis processes into three functionally distinct yet closely collaborative layers—planning, execution, and review—the GeoAgent framework achieves specialized processing at each stage while maintaining high flexibility. This hierarchical multi-agent collaborative mechanism not only effectively overcomes the limitations of single agents in handling complex spatial analysis tasks but also improves system explainability and controllability through clear functional division.

Based on current experimental results and technological development trends, we believe this framework can be further developed in three directions: First, there is considerable room for improving the intelligence level of agents at each layer. Taking the Analyst agent in the Execution Layer as an example, its generate-then-revise code generation strategy can be further optimized by exploring more efficient iterative strategies and internal validation mechanisms. Concurrently, directly integrating pre-trained code models specifically fine-tuned for the geospatial domain is an important path to achieving a leap in its capabilities. Second, the assessment of geospatial cartographic results by the Image Reviewer in the Review Layer currently relies primarily on the multimodal perception capabilities of LLMs. However, research concerning the deep understanding, adherence to professional standards, and aesthetic evaluation of geospatial visualizations by LLMs is still insufficient. Third, collaboration mechanisms between agents at different levels still have room for improvement, especially in handling highly dynamic spatial analysis tasks. How to achieve more flexible and adaptive interlayer interaction and information sharing will be key to enhancing overall system performance.

## 6. Conclusion

The wave of LLMs is driving the rapid evolution of GIS towards intelligence. To address the challenges of automation, intelligence, and result reliability in complex spatial analysis tasks, this paper introduces GeoAgent, an innovative hierarchical multi-agent collaborative framework. GeoAgent achieves in-depth parsing of user requirements and task decomposition through its meticulously designed Planning Layer; its Execution Layer integrates specialized coding, cartography, and reporting agents for efficient collaborative work; and its Review Layer introduces a multi-dimensional assessment and feedback mechanism, enabling autonomous verification and iterative optimization of the spatial analysis process. Experimental results on a comprehensive test set covering 147 tasks of varying types and complexities demonstrate that GeoAgent achieved an overall task success rate as high as 95.24% under the DeepSeek-V3 model configuration. Furthermore, ablation studies clearly revealed the core value of the Planning Layer for effective task decomposition and the Review Layer for ensuring output quality and achieving error correction. Under the configuration of the GPT-4o model, their absence led to significant decreases in success rate of approximately 35.38% and 25.17%, respectively. The case studies further validated GeoAgent's practical effectiveness and application potential in handling complex geospatial decision-making problems.

Despite GeoAgent demonstrating encouraging performance and potential in automated spatial analysis, we recognize that there is still ample room for improvement in areas such as simulating the depth of complex spatial reasoning characteristic of top human experts, handling ultra-large-scale or highly dynamic geographic data, and achieving autonomous learning and evolution of the agents. Future research will focus on continuously enhancing the professional skills and cognitive intelligence of each agent, exploring more efficient and adaptive inter-agent collaboration paradigms, expanding the coverage and challenge level of evaluation benchmarks, and performing architectural optimization and functional iteration of the existing framework.

In conclusion, GeoAgent contributes an empirically validated, modular LLM-based agent system design method with critical self-verification capabilities to the field of geospatial artificial intelligence. The hierarchical multi-agent framework validated by GeoAgent in this study provides a concrete technical pathway and valuable practical reference for developing more robust and intelligent automated spatial analysis tools, and positively contributes to advancing autonomous GIS systems towards a more mature stage in the processing of specific tasks.

## Disclosure statement

No potential conflict of interest was reported by the author(s).

## Funding

This work is supported by the Natural Science Foundation of Zhejiang Province [No. ZCLMS26D0105], the National Natural Science Foundation of China [No.32471860], the National Natural Science Foundation of China [No.42571529], and Deep-time Digital Earth (DDE) Big Science Program.

## Notes on contributors

**Qingming Lin** is a graduate student at Zhejiang Agriculture and Forestry University. His research interests include GeoAI and large language models. His main contributions to the paper are methodology, software, visualization and writing-original draft.

**Liuchang Xu** received his Ph.D. degree in remote sensing and geographic information system from Zhejiang University, China, in 2020 and is currently an Associate Professor with the College of Mathematics and Computer Science, Zhejiang Agriculture and Forestry University, China. His research interests include spatiotemporal analytics, artificial intelligence, urban computing, signal processing, and big data mining. His main contributions to the paper are conceptualization, project administration and writing-review and editing.

**Sensen Wu** received his Ph.D. degree in cartography and geographic information system from Zhejiang University, Hangzhou, China, in 2018 and is currently a Researcher in the School of Earth Sciences, Zhejiang University. His current research interests include spatiotemporal analysis, remote sensing, and deep learning. His main contributions to the paper are funding acquisition and writing-review and editing.

**Ruichen Mao** received his Ph.D. degree in cartography and geographic information system from Zhejiang University, Hangzhou, China, in 2019 and is currently with Moganshan Geospatial Information Laboratory, Huzhou, China. His main contributions to the paper are formal analysis and writing-review and editing.

**Chao Wang** received her Ph.D. degree from Zhejiang University, Hangzhou, China, in 2021 and is currently a Researcher in the School of Earth Sciences, Zhejiang University. Her research interests include spatiotemporal data mining, geospatial big data modeling, and agent-based social system simulation. Her main contributions to the paper are investigation and writing-review and editing.

**Hailin Feng** received his Ph.D. degree from the University of Science and Technology of China and is currently a Professor in the College of Mathematics and Computer Science, Zhejiang Agriculture and Forestry University. His main research interests include artificial intelligence, computer vision, and the Internet of Things. His main contributions to the paper are supervision and funding acquisition.

**Bo Huang** is currently a Chair Professor in the Department of Geography, The University of Hong Kong. His research interests include spatiotemporal statistics, unified satellite image fusion, and multi-objective spatial optimization. His main contributions to the paper are supervision and writing-review and editing.

**Zhenhong Du** is currently a Professor with the School of Earth Sciences, the Dean of the School of Earth Sciences, and the Director of the Institute of Geography and Spatial Information and Zhejiang Provincial Key Laboratory of Geographic Information System. His research interests include remote sensing and geographic information science, spatiotemporal big data and artificial intelligence. His main contributions to the paper are resources and supervision.

## ORCID

Qingming Lin  <http://orcid.org/0009-0006-5875-8676>

Liuchang Xu  <http://orcid.org/0000-0001-7635-7266>

Sensen Wu  <http://orcid.org/0000-0001-9322-0149>

Bo Huang  <http://orcid.org/0000-0002-5063-3522>

Zhenhong Du  <http://orcid.org/0000-0001-9449-0415>

## Data and codes availability statement

The data and codes that support the findings of this study are available at <https://doi.org/10.6084/m9.figshare.29145008>.

## References

- Achiam, J., et al., 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*. Available from: <https://arxiv.org/abs/2303.08774>
- Akinboyewa, T., et al., 2025. Gis copilot: towards an autonomous gis agent for spatial analysis. *International Journal of Digital Earth*, 18 (1), 2497489. <https://doi.org/10.1080/17538947.2025.2497489>
- Chen, M., et al., 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*. Available from: <https://arxiv.org/abs/2107.03374>
- Chen, Y.C., et al., 2025. Enhancing function-calling capabilities in LLMs: Strategies for prompt formats, data integration, and multilingual translation. In: W. Chen, Y. Yang, M. Kachuee, and X.Y. Fu, eds. *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*, April, Albuquerque, New Mexico. Association for Computational Linguistics, 99–111. Available from: <https://aclanthology.org/2025.naacl-industry.9/>.
- Chen, Y., et al., 2024. An llm agent for automatic geospatial data analysis. *arXiv preprint arXiv:2410.18792*. Available from: <https://arxiv.org/abs/2410.18792>
- de Smith, M., Goodchild, P., and Longley, P., 2018. *Geospatial analysis: a comprehensive guide to principles techniques and software tools*. UK: The Winchelsea Press. Available from: <https://www.spatialanalysisonline.com>
- Feng, J., et al., 2024. Citybench: Evaluating the capabilities of large language models for urban tasks. *arXiv preprint arXiv:2406.13945*. Available from: <https://arxiv.org/abs/2406.13945>
- Hong, S., et al., 2024. Metagpt: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*. Available from: <https://arxiv.org/abs/2308.00352>
- Hou, S., et al., 2025. Geocogent: an llm-based agent for geospatial code generation. *International Journal of Geographical Information Science*, 1–34. <https://doi.org/10.1080/13658816.2025.2549460>
- Hou, S., et al., 2024a. Chain-of-programming (cop): Empowering large language models for geospatial code generation. *International Journal of Digital Earth*, 18 (1), 2509812. <https://doi.org/10.1080/17538947.2025.2509812>
- Hou, S., et al., 2024b. Gee-ops: An operator knowledge base for geospatial code generation on the google earth engine platform powered by large language models. *Geo-spatial Information Science*, 1–22. <https://doi.org/10.1080/10095020.2025.2505556>
- Hou, S., et al., 2024c. Can large language models generate geospatial code? *Geo-Spatial Information Science*, 1–35. <https://doi.org/10.1080/10095020.2025.2535523>
- Hou, S., et al., 2024d. Geocode-gpt: A large language model for geospatial code generation tasks. *International Journal of Applied Earth Observation and Geoinformation*, 104456. <https://doi.org/10.1016/j.jag.2025.104456>
- Jiang, J., et al., 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*. Available from: <https://arxiv.org/abs/2406.00515>
- Li, Z., and Ning, H., 2023. Autonomous gis: the next-generation ai-powered gis. *International Journal of Digital Earth*, 16 (2), 4668–4686. <https://doi.org/10.1080/17538947.2023.2278895>
- Lin, Q., et al., 2025. Shapefilegpt: a multi-agent large language model framework for automated shapefile processing. *International Journal of Digital Earth*, 18 (2), 2577884. <https://doi.org/10.1080/17538947.2025.2577884>
- Liu, A., et al., 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. Available from: <https://arxiv.org/abs/2412.19437>
- Luo, Q., et al., 2026. Geojson agents: a multi-agent llm architecture for geospatial analysis—function calling vs. code generation. *Big Earth Data*, 1–55. <https://doi.org/10.1080/20964471.2026.2615511>

- Mai, G., et al., 2025. Towards the next generation of geospatial artificial intelligence. *International Journal of Applied Earth Observation and Geoinformation*, 136, 104368. <https://doi.org/10.1016/j.jag.2025.104368>
- Ning, H., et al., 2025. An autonomous gis agent framework for geospatial data retrieval. *International Journal of Digital Earth*, 18 (1), 2458688. <https://doi.org/10.1080/17538947.2025.2458688>
- Qu, C., et al., 2025. Tool learning with large language models: A survey. *Frontiers of Computer Science*, 19 (8), 198343. <https://doi.org/10.1007/s11704-024-40678-2>
- Simon, H., 1997. Models of bounded rationality: Empirically grounded economic reason. *Models of bounded rationality*. Cambridge, MA: MIT Press. Available from: <https://books.google.com.sg/books?id=9CiwU28z6WQC>.
- Team, G., et al., 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*. Available from: <https://arxiv.org/abs/2312.11805>
- Touvron, H., et al., 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*. Available from: <https://arxiv.org/abs/2302.13971>
- Wang, L., et al., 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18 (6), 186345. <https://doi.org/10.1007/s11704-024-40231-1>
- Wei, C., et al., 2025. Geotool-gpt: a trainable method for facilitating large language models to master gis tools. *International Journal of Geographical Information Science*, 39 (4), 707–731. <https://doi.org/10.1080/13658816.2024.2438937>
- Wu, H., et al., 2025. Geocolab: an llm-based multi-agent collaborative framework for geospatial code generation. *International Journal of Digital Earth*, 18 (2), 2569405. <https://doi.org/10.1080/17538947.2025.2569405>
- Wu, Q., et al., 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*. Available from: <https://arxiv.org/abs/2308.08155>
- Xi, Z., et al., 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68 (2), 121101. <https://doi.org/10.1007/s11432-024-4222-0>
- Xu, L., et al., 2025. Evaluating large language models on geospatial tasks: A multiple geospatial task benchmarking study. *International Journal of Digital Earth*, 18 (1), 2480268. <https://doi.org/10.1080/17538947.2025.2480268>
- Yao, S., et al., 2023. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*. Available from: <http://arxiv.org/abs/2210.03629>
- Zhang, Q., et al., 2025. Geoanalystbench: A geoai benchmark for assessing large language models for spatial analysis workflow and code generation. *Transactions in GIS*, 29 (7), e70135. 8156745. <https://doi.org/10.1111/tgis.70135>
- Zhang, Y., et al., 2024a. Mapgpt: an autonomous framework for mapping by integrating large language model and cartographic tools. *Cartography and Geographic Information Science*, 51 (6), 717–743. <https://doi.org/10.1080/15230406.2024.2404868>
- Zhang, Y., et al., 2024b. Geospatial large language model trained with a simulated environment for generating tool-use chains autonomously. *International Journal of Applied Earth Observation and Geoinformation*, 136, 104312. <https://doi.org/10.1016/j.jag.2024.104312>
- Zhang, Y., et al., 2024c. Geogpt: An assistant for understanding and processing geospatial tasks. *International Journal of Applied Earth Observation and Geoinformation*, 131, 103976. <https://doi.org/10.1016/j.jag.2024.103976>

## Appendix A.

### System implementation details

#### A.1. Planner workflow and plan templates

To further illustrate the internal mechanism of the Planning Layer, this section provides the workflow diagram of the Planner. As shown in Figure A1, the Planner consists of two core

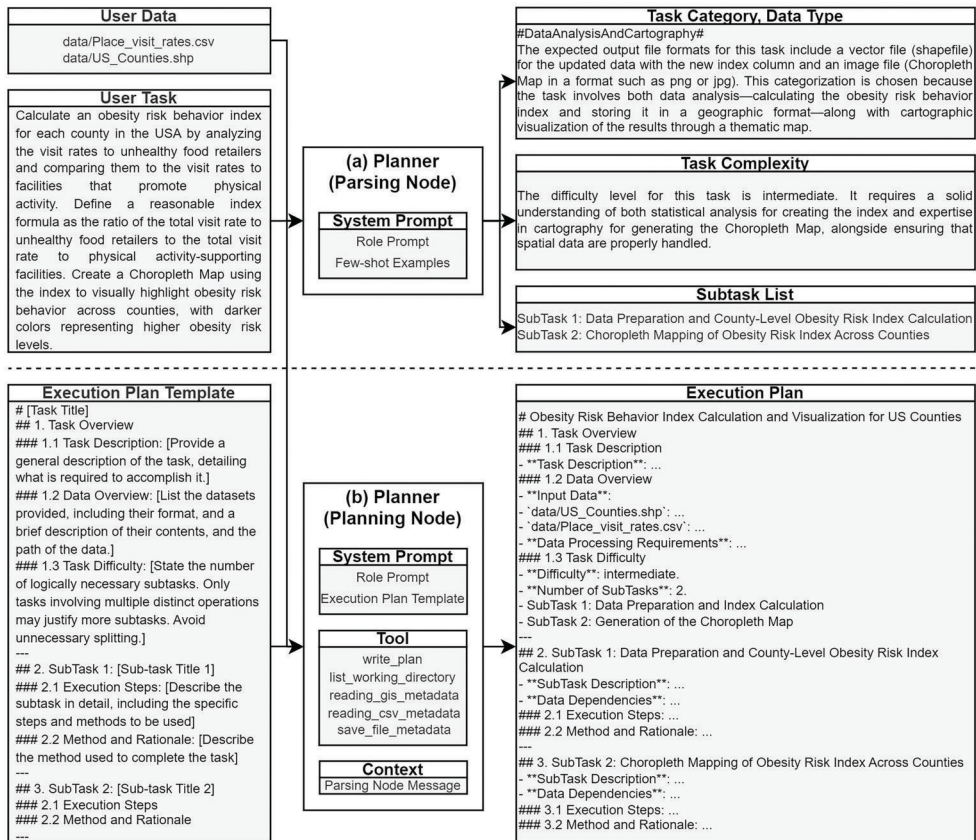


Figure A1. Workflow of the planner within the planning layer.

#### Markdown Template for Task Execution Plan

```
# [Task Title]
## 1. Task Overview
### 1.1 Task Description
- **Task Description***: [Provide a general description of the task, detailing what is required to accomplish it. This could include objectives, expected outcomes, or specific actions that need to be taken.]
### 1.2 Data Overview
- **Input Data***: List the datasets provided, including their format (e.g., Shapefile, Tiff), and a brief description of their contents (e.g., candidate_landfill.xls contains potential landfill locations), and the path of the data (e.g., data/landfill/candidate_landfill.xls.)
- **Data Processing Requirements***: Mention any preprocessing needed (e.g., coordinate system alignment, data cleaning, table to vector, etc.)
### 1.3 Task Difficulty
- **Difficulty***: [Assess the complexity of the task. Is it a straightforward process or does it require advanced techniques or domain-specific knowledge? You can categorize it as low, medium, or high difficulty based on factors such as data quality, tools required, and technical expertise needed.]
- **Number of SubTasks***: [State the number of logically necessary subtasks. Simple or moderate tasks should typically be completed in one subtask, and should not exceed two subtasks under any circumstances. Only tasks involving multiple distinct operations may justify more subtasks. Avoid unnecessary splitting such as separating filtering and saving into separate subtasks.]
---
## 2. SubTask 1: [Sub-task Title 1]
- **SubTask Description***: [Describe the subtask in detail, including the specific steps and methods to be used].
```

```
### 2.1 Execution Steps:
1. **Step 1**:. [Describe the first step]. (e.g., Utilize the provided coordinate columns (Latitude and Longitude)
to construct geometric points.)
2. **Step 2**:. [Describe the next step]. (e.g., Apply a datum transformation and re-projection to align them
with the target projected CRS.)
3. **Step 3**:. [Describe the next step]. (e.g., Save the transformed point geometry and its associated attributes
into a shapefile.)
### 2.2 Method and Rationale:
- **Method**:. [Describe the method used to complete the task]. (e.g., Feature Construction based on Cartesian
coordinates, followed by a Geodetic Transformation to establish a suitable projected coordinate system.)
- **Rationale**:. [Explain why this method was chosen]. (e.g., The data must transition from a non-spatial table
to a georeferenced vector dataset to enable spatial query and analysis operations, and coordinate reprojection
ensures consistency with other datasets.)
```

components: task parsing and task planning. The task parsing component generates a task description that includes information such as task category, data type, and difficulty level. Based on this parsed context, the task planning component constructs the execution plan. The workflow diagram demonstrates how the Planner extracts key guiding information from the input task and produces a readable and structured execution plan within a predefined template.

Given the central role of the execution plan within the overall execution pipeline, this section provides the Markdown template used by the Planner. The template consists of fixed fields such as task overview, input data description, subtask sequence, and output requirements. The predefined structure reduces uncertainty in both the format and semantics of the planning results, enabling the Manager to accurately interpret subtask boundaries, dependencies, and constraints based on consistent field identifiers.

A.2. Execution control parameters

The stable operation of the framework relies on several execution control parameters, including the maximum subtask revision count and the global execution step limit. Table A1 summarizes these parameters, their assigned values, the components they apply to, and their functional roles within the system. Based on the main experimental results in Table 3 and the efficiency evaluation in Table 4, the maximum subtask revision count is set to 1 in this study to balance computational efficiency with local error-correction capability. The global execution step limit constrains the total number of agent actions and prevents infinite loops that may occur when the system repeatedly cycles through unresolved subtasks or fails to reach a terminating condition. For the spatial analysis tasks considered in this work, setting the step limit to 32 is sufficient to cover the full execution range and allows the Manager agent to complete approximately six cycles of subtask scheduling. It should be noted that increasing the revision count for more complex scenarios would also require raising the global execution step limit to ensure that the execution process can complete and terminates properly under deeper iterative conditions.

Table A1. Execution control parameters used in GeoAgent.

| Parameter                   | Symbol    | Default Value | Description                                                                                                              |
|-----------------------------|-----------|---------------|--------------------------------------------------------------------------------------------------------------------------|
| Global execution step limit | $N_{max}$ | 32            | Maximum number of agent execution steps permitted per task, used to prevent infinite loops or excessive execution depth. |
| Subtask revision limit      | $R_{max}$ | 1             | Maximum number of local revision iterations allowed for each subtask before it is finalized or marked as failed.         |

## Appendix B.

### Evaluation protocol and comparative context

#### B.1. Manual evaluation criteria and checklist

When executing spatial analysis tasks, the framework produces multiple types of outputs, including vector data, raster data, statistical figures, and textual reports. The evaluation procedure must therefore accommodate these heterogeneous outputs. To enhance the objectivity and reproducibility of the assessment process, this section provides a structured human evaluation checklist. As shown in Table B1, the checklist specifies clear verification requirements for spatial data, including format validity, consistency of coordinate reference systems, semantic appropriateness of geometry types, and correctness of key attribute fields. For visualization outputs, the checklist further outlines criteria concerning layer presentation, representation of spatial relationships, and the completeness and accuracy of auxiliary elements. The evaluation of textual outputs focuses on the semantic correctness of conclusions and the extent to which they address the task instructions. The checklist follows a binary decision scheme, and a task is considered successfully completed only when all evaluation items are satisfied.

**Table B1.** Checklist for manual evaluation of geospatial task outputs.

| Requirement                                                                                                                                                                                                                                                    | Yes                      | No                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|--------------------------|
| <b>Geospatial Data Files</b>                                                                                                                                                                                                                                   |                          |                          |
| 1. Spatial reference correctness: The Coordinate Reference System is explicitly defined, matches the task specifications, and aligns with the input data.                                                                                                      | <input type="checkbox"/> | <input type="checkbox"/> |
| 2. Data structure and type validity: The data format accurately represents the spatial entity: for vector data, the geometry type is semantically correct; for raster data, the spatial resolution, grid dimensions, and band count align with analysis needs. | <input type="checkbox"/> | <input type="checkbox"/> |
| 3. Attribute/value completeness: Required attribute fields (for vectors) or pixel value ranges/statistics (for rasters) are present, meaningful, and consistent with the task demands.                                                                         | <input type="checkbox"/> | <input type="checkbox"/> |
| 4. Spatial constraint compliance: The output satisfies the spatial conditions required by the task, such as correct spatial filtering, region constraints, buffer distances, overlay results, or raster computation outcomes.                                  | <input type="checkbox"/> | <input type="checkbox"/> |
| <b>Visual Outputs</b>                                                                                                                                                                                                                                          |                          |                          |
| 1. Data correctness: The visualization is constructed from correct data sources and reflects the underlying analytical results accurately; mapped values or categories correspond to computed statistics.                                                      | <input type="checkbox"/> | <input type="checkbox"/> |
| 2. Symbolization logic soundness: The visual encoding (e.g., color schemes, symbol sizes, line weights) effectively differentiates spatial patterns or categories and avoids misleading representations.                                                       | <input type="checkbox"/> | <input type="checkbox"/> |
| 3. Essential map elements completeness: The visualization includes key cartographic elements necessary for interpretation, such as a legend, title, and relevant labels, ensuring that it can be understood independently.                                     | <input type="checkbox"/> | <input type="checkbox"/> |
| <b>Analytical Reports</b>                                                                                                                                                                                                                                      |                          |                          |
| 1. Core question coverage: The report directly addresses the analytical objectives specified by the task.                                                                                                                                                      | <input type="checkbox"/> | <input type="checkbox"/> |
| 2. Analytical validity: The conclusions presented in the report are supported by the correct analytical procedures and consistent with the derived geospatial results.                                                                                         | <input type="checkbox"/> | <input type="checkbox"/> |

**Table B2.** Comparison of GeoAgent with representative LLM-based and agent-based geospatial frameworks, summarized across system type, task type, evaluation methods, and evaluation metrics.

| Study                                       | System Type           | Task Type        | Evaluation Methods                        | Evaluation Metrics                                                 |
|---------------------------------------------|-----------------------|------------------|-------------------------------------------|--------------------------------------------------------------------|
| GeoTool-GPT (Wei <i>et al.</i> 2025)        | Fine-tuned LLM        | Tool use         | Human evaluation;<br>Case study           | Accuracy                                                           |
| GeoCode-GPT (Hou <i>et al.</i> 2024d)       | Fine-tuned LLM        | Code generation  | Human evaluation;<br>Objective evaluation | Accuracy;<br>Completeness;<br>Readability;<br>Executability        |
| GIS Copilot (Akinboyewa <i>et al.</i> 2025) | LLM-based agent       | Spatial analysis | Human evaluation;<br>Case study           | Success rate                                                       |
| ShapefileGPT (Lin <i>et al.</i> 2025)       | LLM-based multi-agent | Spatial analysis | Human evaluation;<br>Case study           | Accuracy; Success rate;<br>Efficiency                              |
| GeoCogent (Hou <i>et al.</i> 2025)          | LLM-based agent       | Code generation  | Human evaluation;<br>Case study           | Executability;<br>Correctness;<br>Readability;<br>Efficiency; Cost |
| GeoColab (Wu <i>et al.</i> 2025)            | LLM-based multi-agent | Code generation  | Human evaluation;<br>Case study           | Executability;<br>Accuracy;<br>Readability;<br>Efficiency; Cost    |
| GeoJsonAgent (Luo <i>et al.</i> 2026)       | LLM-based multi-agent | Spatial analysis | Human evaluation;<br>Case study           | Accuracy; Efficiency                                               |
| GeoAgent (Ours)                             | LLM-based multi-agent | Spatial analysis | Human evaluation;<br>Case study           | Success rate;<br>Efficiency; Cost                                  |

## B.2. Comparison with existing frameworks

To contextualize the proposed framework within ongoing research on large language model-driven autonomous GIS systems, this section provides a comparative summary highlighting its relationships and distinctions with several representative existing approaches. The comparison considers dimensions such as framework type, task type, evaluation methodology, and evaluation metrics. Existing methods vary in their technical focus and task objectives: some primarily aim to enhance foundational model capabilities in code generation or tool invocation, while others investigate agent structures to support spatial processing workflows. In terms of evaluation, most approaches still rely on expert review and case-based validation. GeoAgent is designed to support end-to-end execution within real geospatial computing environments, and its performance is assessed using indicators such as task success rate, execution efficiency, and computational cost, thereby reflecting its effectiveness across complete spatial analysis workflows.